MOBILE LLM INFERENCE AND INTELLIGENT SELECTION - ENERGY AWARENESS

PHASE I REPORT

Submitted by

MONIC AUDITYA A	**230701194**
ATCHAYA S	**230701045**
RAMYA SREEVARSHINI B	**230701262**
SARANYA M	**230701510**

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING





RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

DECEMBER 2026

BONAFIDE CERTIFICATE

Certified that this Report titled "**MOBILE LLM INFERENCE AND INTELLIGENT SELECTION - ENERGY AWARENESS**" is the bonafide work of **Monic Auditya A (230701194), Atchaya S (230701045), Ramya Sreevarshini B (230701262), Saranya M (230701510)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

"This project addresses the following Sustainable Development Goal(s): **SDG 9 : Industry, Innovation and Infrastructure.**"

SIGNATURE

Dr. J. Manoranjini,
Professor and Head,
Department of Computer Science and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

Karthikeyan N,
Assistant Professor,
Department of Computer Science and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on

_____

Internal Examiner **External Examiner**

ABSTRACT

As more and more large language models are deployed on smartphones, this raises the possibility to offer AI chat services over a device without internet connection. But smartphones have different processing capabilities, memory, battery status, thermal condition, supported models, and existing loads. Choosing a dedicated device, or rotating requests from model to model naively ignores those factors. Likewise choosing models naively without the awareness of its resource footprint and switching costs leads to additional delay and energy consumption.

This report proposes an intelligent joint routing architecture for Offline Group AI Chat on local Wi-Fi. It leverages device-aware routing and multi-model routing to intelligently pick the right host phone and language model for processing each query. Queries are aggregated based on a lightweight LCI of all local devices, and query complexity, model performance, memory, thermals, battery state, q-length, and network performance is used for model & host selection. A swap-cost-aware stay/switch policy with hysteresis prevents excessive host switching.

This architecture has meshes to allow and satisfy control, data collection, LCI calculation, event routing, workload-to-query complexity estimation, control plane selection multi-models, with swap-cost analysis, local inference, dashboard monitoring. Our design goal is transparent decision, local calculation, explicit recovery, user control. Our design and implementation and evaluation includes Quality, latency, maximum throughput, switching cost, energy, thermal, reliability, scalability with various workloads.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E., F.I.E.,** our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S. N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. Manoranjini, Ph.D.,** Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Karthikeyan N,** Assistant Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

Monic Auditya A (230701194)

Atchaya S (230701045)

Ramya Sreevarshini B (230701262)

Saranya M (230701510)

TABLE OF CONTENTS

ABSTRACT ..... **iii**

LIST OF TABLES ..... **vii**

LIST OF FIGURES ..... **viii**

LIST OF ABBREVIATIONS ..... **ix**

1. INTRODUCTION TO DEVICE-AWARE ROUTING FOR OFFLINE GROUP AI CHAT **1**

1.1 GENERAL 1

1.2 OBJECTIVES 2

1.3 EXISTING SYSTEM 3

1.4 PROPOSED SYSTEM 5

1.5 SCOPE OF THE PROJECT 6

1.6 ORGANIZATION OF THE REPORT 7

2. LITERATURE SURVEY ON MOBILE LLM INFERENCE AND SCHEDULING **8**

2.1 LITERATURE REVIEW 8

2.2 COMPARISON AND DISCUSSION 14

2.3 CONCLUSION 17

3. PROPOSED SYSTEM **18**

3.1 SYSTEM ARCHITECTURE 18

3.2 SYSTEM REQUIREMENTS 21

3.2.1 SOFTWARE REQUIREMENTS 21

3.2.2 HARDWARE REQUIREMENTS 23

3.2.3 SECURITY REQUIREMENTS 24

3.2.4 DEPLOYMENT AND SCALING 26

3.3 SYSTEM DESIGN 27

3.3.1 ACTIVITY DIAGRAM 27

3.3.2 DATA FLOW DIAGRAM 29

4. PROJECT DESCRIPTION.....31
4.1 TECHNOLOGIES AND METHODOLOGY.....31
4.2 MAIN MODULES OF THE PROJECT.....32
4.2.1 MESH COMMUNICATION MODULE.…32
4.2.2 LCI COMPUTATION MODULE.....33
4.2.3 EVENT-BASED ROUTING MODULE.....36
4.2.4 HOST INFERENCE MODULE.....39
4.2.5 DASHBOARD.....41
5. IMPLEMENTATION AND RESULTS.....42
5.1 IMPLEMENTATION RESULTS.....42
5.1.1 TELEMETRY COLLECTION AND PREPROCESSING.....42
5.1.2 IMPLEMENTATION OF LCI COMPUTATION.....44
5.1.3 IMPLEMENTATION OF EVENT-BASED ROUTING.....48
5.1.4 DASHBOARD AND APPLICATION SCREENSHOTS.....50
5.2 EVALUATION AND RESULT DISCUSSION.....52
6. CONCLUSION AND FUTURE WORK.....56
6.1 CONCLUSION.....56
6.2 FUTURE WORK.....57
REFERENCES.....59

LIST OF TABLES

2.1 Comparison of existing works.....15

3.1 Software requirements of the proposed system.....22

3.2 Hardware requirements of the proposed system.....24

4.1 LCI signals, orientation and weights.....35

4.2 Elective and forced routing triggers.....37

4.3 Prepare–ready–route handoff messages.....39

5.1 Device configuration in the five-phone evaluation.....43

5.2 Inference throughput and request latency.....46

5.3 Tail latency, thermal metric and request success.....46

5.4 Time to first token across prompt lengths.....49

5.5 Prefill throughput across prompt lengths.....49

5.6 Endurance, battery temperature and failure outcomes…53

5.7 Handoff outcomes and availability statistics.....53

LIST OF FIGURES

1.1 Overview of the Proposed System.....5

3.1 System Architecture, Phone Roles over the Local Mesh19

3.2 Activity Diagram for the Router Environment.....28

3.3 Activity Diagram for the Host Environment.....29

3.4 Data Flow Diagram of the System.....30

4.1 LCI Telemetry and Candidate Scoring Pipeline.....35

4.2 Event-Based Routing State Machine.....38

5.1 Thermal Degradation and Energy per Generated Token44

5.2 Sustained Throughput and Battery-Temperature Dynamics during the Continuous Session.....47

5.3 Request-Latency Cumulative Distribution and Tail Percentiles.....50

5.4 HAYA Application Access and Device Configuration..51

5.5 HAYA Chat Output and Group Interaction.....52

5.6 Multi-Dimensional Performance Trade-off Analysis.....54

LIST OF ABBREVIATIONS

Abbreviation	Expansion
ABI	Application Binary Interface
API	Application Programming Interface
CPU	Central Processing Unit
DRL	Deep Reinforcement Learning
DVFS	Dynamic Voltage and Frequency Scaling
GPU	Graphics Processing Unit
HAL	Hardware Abstraction Layer
IPC	Inter-Process Communication
JNI	Java Native Interface
LCI	Live Compute Index
LLM	Large Language Model
MEC	Mobile Edge Computing
MTBF	Mean Time Between Failures
NDK	Native Development Kit
NPU	Neural Processing Unit
OOM	Out of Memory
OS	Operating System
RAM	Random Access Memory
RTT	Round Trip Time
SoC	System on Chip
TAS	Temperature-Aware Scheduling
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TTFT	Time to First Token
UI	User Interface
UML	Unified Modeling Language
Wi-Fi	Wireless networking technology

CHAPTER 1
INTRODUCTION TO DEVICE-AWARE ROUTING FOR OFFLINE GROUP AI CHAT

1.1 GENERAL

The existence of compact models on smartphones offers the chance to collaborate with AI assistance using a local group of devices, which will not rely on servers or even stable internet connection. Local group sessions will allow collaboration in learning, discussion, and assistance during low-connectivity periods. Inference inside the local trusted network will make users independent from any additional services and give them more control over the location where the conversation will be processed.

The capability of running models on phones does not mean that all phones have the same computational capabilities. Phones may vary in chip version, memory capacity, supported models, cooling, battery level, and ongoing activity. The phone suitable for inference at the beginning of the session may become unsuitable for further inference due to repeated generation requests. On the other hand, the phone busy at the beginning of the session might become available after some time. Static host assignment and random rotation will yield inconsistent response times. The phone with low memory may not pass an admission check or run out of memory. The high performance phone may slow down due to prolonged overheating. Frequent change of hosts may lead to loss of runtime state and reconstruction of conversation context on the new host. The routing algorithm should

take into account both benefits of choosing a more suitable device and costs of switching sessions.

The described system considers host choice as a continually observed decision instead of the outcome of benchmarks. Telemetry provides lightweight information about the current state of each phone, and the event-based policy determines when the currently selected host should be replaced. The goal is to provide a responsive and explainable service for local group chats with explicit constraints on resource usage and specific behavior when there is no suitable host found. Throughout this report, the word mesh stands for a logical network of the collaborating peers, which are phones in our case. We do not assume the implementation of a multi-hop Wi-Fi mesh on regular Android phones.

1.2 OBJECTIVES

The overall goal is to build real-time, device-aware routing of AI chat queries amongst a set of locally connected phones. Selection of the host should be based on the availability of compute resources, the compatibility of the model and stability rather than some single ranking of hosts. The detailed goals are to:

1. Identify trusted participating phones and keep an up-to-date registry of device capability, available models and hosting role.
2. Design an LCI that takes into account decode latency, memory margin, thermal margin, queue availability, battery status and network RTT in an open score.

3. Use admission filters prior to ranking hosts such that high scores do not override incompatible models, unsafe conditions and insufficient memory.
4. Reconsider host assignments when device or network events occur, with stickiness to avoid unnecessary handoff.
5. Implement conversation handoff with a prepare/ready/route flow and explicit timeouts, rollback and recovery behaviors.
6. Offer a dashboard that provides details of scores, filtered hosts, host assignments and resource status.
7. Assess performance, energy, reliability and scalability using reproducible workloads and proper baseline comparison.

The suggested design uses a telemetry heartbeat rate of 1 s, a budget for scoring and probing under 80 ms and a target for warm-host routing-control handoff under 200 ms. These numbers represent engineering goals but not demonstrated performance. The time spent on model loading, context reconstruction and reconnect is measured separately.

1.3 EXISTING SYSTEM

Scheduling in existing inference systems for mobile and edge computing takes place at multiple levels of granularity: selecting a processor inside one phone, offloading tasks to an edge server, or splitting one model among multiple devices. These methods solve different instances of the resource-allocation problem, but the granularity of their scheduling and their assumptions of device

availability and state differ significantly from session-wide routing between peer phones. The common approaches are summarized below.

Static host assignment: A single high-capability phone receives all requests. This technique is simple to implement and allows a warm model cache to be maintained, though it does not take into account changes in temperature, battery state or background activity on this device.

Round-robin assignment: Requests cycle through the set of participating devices. This technique distributes requests evenly, but does not consider the ability of each device to allocate the necessary amount of memory for the model or provide acceptable response time. In addition, conversation-state synchronization becomes more costly.

Queue- or latency-based dispatch: The device with the shortest queue or minimal network RTT is selected by the scheduler. These metrics reflect a part of the service cost, but do not represent memory availability or sustained processing capability.

Thermal-aware and battery-aware scheduling: These techniques add information about device state. The works describe relevant techniques, although several operate within one device or use an edge-computing layer. They are discussed in Chapter 2.

Neither of the two extremes is preferable for our setup. Leaving a deteriorating host as a participant without changing the selection will cause deterioration of the user experience, whereas changing the

selection on each query is too costly in terms of switching. Hence, we combine periodic and event-driven approaches with hysteresis to achieve the desired properties.

1.4 PROPOSED SYSTEM

The suggested framework combines mesh communication, an LCI engine, candidate selection and evaluation, an event monitor, handover coordination, and a user interface. Every phone reports its current status to the router, which chooses a single viable node for a whole conversation instead of dividing every new round among several phones.

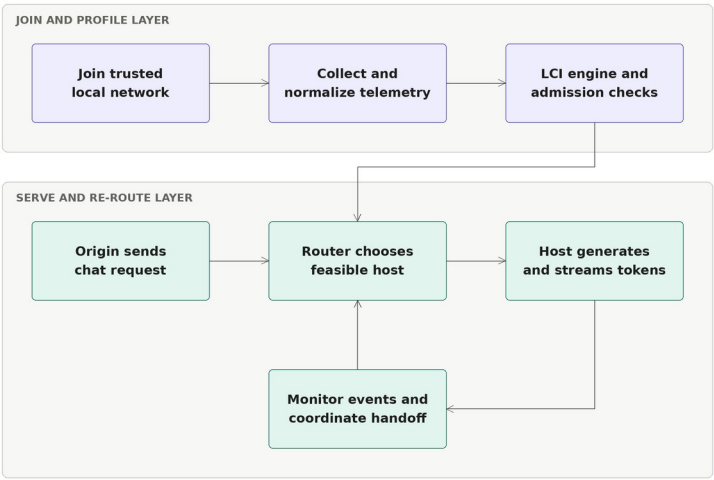


Figure 1.1. Overview of the Proposed System

Figure 1.1 illustrates how the join-and-profile layer differs from the serving-and-re-routing layer. Telemetry is maintained via a proposed heartbeat every second, while the active requests use the most recent

valid device snapshot. Devices joining, leaving, and changing roles do not assume continuous availability of all devices.

LCI is the measure of current suitability of resources. Another condition check is performed on memory availability, model identity, validity, consent, and safety. The event monitor invokes re-selection if the current host becomes unsuitable or if there is another suitable candidate whose suitability level is relatively improved by 8%. An emergency recovery operation does not consider this relative margin when the current host becomes infeasible.

All routing and inference is expected to occur locally. This alone is not sufficient for ensuring privacy – participants need to authenticate each other, protect transport, and recognize that the selected host can see the contents of the conversation. The security constraints in Chapter 3 cover these boundaries.

1.5 SCOPE OF THE PROJECT

The scope of this Phase I work is the design, implementation and preliminary evaluation of whole-conversation routing among a small group of trusted Android phones that share a local Wi-Fi network and have no internet access.

Within scope: discovery and admission of peers; telemetry collection and normalization; the LCI and the admission guards; model-aware host selection; event-based re-evaluation with an 8% relative stickiness margin; prepare/ready/route handoff; local inference with

llama.cpp through JNI on Android; and a dashboard that explains routing decisions.

Outside scope: multi-hop Wi-Fi mesh networking, model partitioning across phones, distributed training, cloud or server-side inference, attestation of peer-reported sensor values, and claims about long-term battery health or population-level reliability. Each of these is either left as future work or is stated explicitly as a limitation.

1.6 ORGANIZATION OF THE REPORT

Chapter 1 introduces the problem, the objectives, the existing systems, the proposed system, the scope of the project and the organization of the report. Chapter 2 reviews the related work on thermal-aware scheduling, offloading, distributed mobile inference and peer-to-peer runtimes, and compares it with whole-session host routing. Chapter 3 presents the system architecture, the software, hardware and security requirements, the deployment strategy, and the activity and data flow diagrams. Chapter 4 describes the technologies and the five modules of the system: mesh communication, LCI computation, event-based routing, host inference and the dashboard. Chapter 5 reports the implementation, the evaluation set-up and the results of the Phase I experiments. Chapter 6 concludes the report and outlines the future work.

CHAPTER 2
LITERATURE SURVEY ON MOBILE LLM INFERENCE AND SCHEDULING

This chapter reviews the research themes related to the proposed scheduling problem. The comparison distinguishes processor scheduling, task offloading, distributed model execution, and whole-session host routing. Publication results are evidence for their own experimental settings, not measured outcomes of this project.

2.1 LITERATURE REVIEW

Thermal-aware scheduling on mobile devices. Tan and Cao [1] examine thermal-aware scheduling of deep neural networks on mobile devices equipped with GPUs and NPUs. They develop a heuristic scheduler and its reinforcement-learning based extension to take future consequences of scheduling into account. The work justifies the consideration of thermal state rather than speed only. The scheduling scope of that work is the processing resources in one mobile device. The choice between independently owned devices and moving an ongoing conversation is outside its scope. For the proposed system, the relevant principle is that instant throughput is not an adequate metric of sustainable service. Thermal state must affect both candidate ranking and hard safety exclusions. The project incorporates the principle without presuming the applicability of the policy presented in the study to inter-device routing.

Temperature-aware heterogeneous computing. Gao's TAS framework [2] examines temperature-aware scheduling of tasks for heterogeneous computing resources. The literature summary mentions a temperature perception model, speed control, task scheduling, and task assignment optimization for CPU, GPU, and NPU resources. The case is relevant due to the consideration of thermal constraints in scheduling. The difference is in the scheduling unit. TAS concerns heterogeneous processors and their task allocation, while this project chooses a particular physical phone that supports the requested model. Thus, the proposed router takes into account such factors as reachability of peers, battery contribution limitations, and preparedness of conversations along with information about local resources.

Thermal-aware single-layer parallelization. Zhang et al. [3] explore thermal-aware dynamic frequency adjustment and heterogeneous single-layer workload distribution. Their approach regulates the speed of processor execution and distributes the workload of the neural network depending on the processor and layer properties. The study concentrates on regulation of on-device execution, including cases when ambient temperature is important. The work shows the feasibility of distinguishing between the sustained capability of a host and its nominal hardware specifications. However, the proposed project does not distribute an individual neural-network layer between phones. The only decision is the destination of a complete conversation – the current host or another feasible device.

Multi-DNN processor co-execution. Gao et al. [4] present an advanced multi-DNN model scheduling approach for heterogeneous mobile processors. Their method combines offline subgraph partitioning with a processor-state-aware runtime algorithm to distribute workload in a changing execution environment. It addresses the problems caused by scheduling complexity and memory overhead in heterogeneous co-execution. The relevance to the project is in the use of the live state of hosts instead of an immutable capacity label. The difference is that the proposed mesh router works at the level of device and conversation. It does not have to reproduce subgraph partitioning from the paper but ensures that each selected host is able to execute the requested model independently.

Sustained LLM inference at the edge. Tummalapalli et al. [5] characterize LLM inference on mobile, NPU, and GPU platforms under sustained load. Their study explores the throughput, latency, power, and thermal performance for a fixed model and workloads. The authors pay special attention to the fact that results represent the performance of both hardware and software rather than a general ranking of hardware capability. The results are directly relevant to the design of the evaluation. A cold-start benchmark is insufficient to characterize a long group chat session. Measurements have to include repetitive requests, warm execution, thermal dynamics, and energy consumption per generated token. Improvement of the proposed system must be established on its own testbed, not by inference from the behavior of another platform.

Network-edge LLM inference survey. Chen et al. [6] review network-edge inference for large language models, including architectures, model optimization, deployment, resource management, and scheduling. The review places the proposed project in the context of a research area driven by constraints in compute, memory, and communication resources. The project tackles a more limited practical situation: a group of trusted phones, a local network without internet access, compatible independently executable models, and transparent whole-conversation routing. This is the focus of the design, not a proof that there are no similar systems. A comprehensive novelty assessment would require a more extensive systematic review than the papers considered here.

LinguaLinked: distributed mobile LLM inference. Zhao et al. [7] introduce LinguaLinked, a decentralized mobile LLM inference system. It divides the model, assigns model segments to devices according to their capabilities, optimizes transmission between segments, and includes runtime load balancing. It allows collaborative execution of the model if it cannot be executed conveniently on one device. The proposed project assumes that at least one phone in the local network can execute the requested model independently. It routes a complete conversation to such a phone and considers a change of host as a handoff. Model parallelization and host selection are two different approaches with distinct network costs and failure modes. A failure of a segment in distributed execution can affect the

whole chain, while in a host-routing design the system has to re-establish or transfer the conversation to another independent executor.

Deep reinforcement learning for offloading. Lim et al. [8] propose DRL-OS, an offloading scheduler based on a double dueling deep Q-network. It uses task size, deadline, queue state, and residual battery charge to select local execution, remote offloading, or dropping of a task. The work presents an example of a state-dependent scheduling formulation instead of a fixed local-or-remote policy. For this project, the relevant message is that battery state and queue state affect the suitability of the service. However, using a trained policy introduces an extra step in the development and validation process of the system. The proposed LCI is deliberately transparent and arithmetic-based. Its overhead still has to be measured and should not be assumed to be within the probe budget because of its formula simplicity.

Battery-care resource allocation after disasters. Tang et al. [9] investigate the allocation of battery-care resources and offloading tasks in a post-disaster mobile edge environment. They take into consideration connectivity issues, device-to-device communication, delays, energy usage, and battery degradation using multi-agent optimization. The relevance to the current system is in the consideration of changes in connectivity and limitations of the battery participation. However, the post-disaster MEC setting and the goals of the optimization are distinct from those of the proposed lightweight group-chat host router. The project explicitly handles disconnections

and configurable battery eligibility without claiming that charge is enough to define battery longevity.

Fuzzy priority-aware task scheduling. Lin [10] proposes a fuzzy-based approach to priority-aware task scheduling for mobile edge computing. The system combines greedy auto-scaling offloading with fuzzy scheduling and resource allocation in a multi-tier environment. This is another example of a multi-criteria task scheduling approach. The proposed LCI uses a weighted, normalized score as the criterion without employing a fuzzy rule base. The weighted score is easier to present, but the process of weighting and normalizing needs calibration. Transparency does not reduce the need for workload-specific evaluation.

Peer-to-peer inference runtimes. Mesh LLM [11] describes distributed inference across networked machines and provides tools for sharing models. Although it is a helpful example of networked inference, its application should not be interpreted as an experiment with Android smartphones. The decision to use whole-session routing in the proposed project allows avoiding sending each generated token across the sequence of partitions in the model. Whether this is beneficial depends on the model, host hardware, network, and handoff frequency. Throughput values reported for such runtimes are not reproduced here as smartphone-mesh results because the project has no matching benchmark configuration.

Related surveys, measurements and techniques. Broader surveys and measurement studies of mobile and edge LLM inference [12],

[13] describe the resource constraints that motivate this work, while power-efficient and memory-efficient on-device techniques [14]–[17] address the cost of running a model on a single phone. Measurement of the efficiency of LLM applications on mobile devices [18] reinforces the need for per-device evaluation. Work on task offloading for collaborative LLM inference [19], edge–cloud surveys [20], and collaborative and decentralized edge inference [21]–[24] studies how several devices can share the inference workload. Earlier research on mobile clouds and computation offloading [25]–[32] provides the foundations for treating nearby phones as shared resources, and work on memory-limited inference, mobile-friendly quantization and smartphone-assistant benchmarking [33]–[35] informs the choice of compact quantized models used here.

2.2 COMPARISON AND DISCUSSION

The reviewed works agree on the relevance of device and workload state information; yet they work at different levels. Processor-based methods optimize execution in one phone. Offloading strategies compare a local device against an external compute layer. Distributed framework models coordinate several devices during one model execution, which is not analogous to selecting one peer phone for the execution of a conversation. Table 2.1 summarizes the comparison.

Table 2.1. Comparison of existing works

Ref.	Scheduling unit / focus	Signals considered	Difference from this work
[1]	DNN layers on mobile GPU/NPU	Thermal state, future effects	One device; no inter-device handoff
[2]	Tasks on CPU/GPU/NPU	Temperature, speed control	Schedules processors, not phones
[3]	Single layers on heterogeneous processors	Frequency, ambient temperature	No inter-phone routing
[4]	Multi-DNN subgraphs	Live processor state, memory overhead	Whole model on each host
[5]	Sustained LLM inference study	Throughput, latency, power, thermals	Measurement study; informs evaluation
[6]	Network-edge LLM inference survey	Compute, memory, communication	Survey; narrower trusted-phone setting
[7]	Model segments across devices	Device capability, transmission, load	Splits the model; we route conversations
[8]	Task offloading (DRL)	Task size, deadline, queue, battery	Learned policy; LCI is arithmetic
[9]	Battery-care offloading (MEC)	Connectivity, energy, battery	Different setting and goals
[10]	Priority-aware task scheduling	Multiple criteria, fuzzy rules	Weighted normalized score
[11]	Distributed inference runtime	Networked machines, model sharing	Not evaluated on smartphones
This work	**Whole conversation: host and model**	**Six-signal LCI plus admission guards**	**Event-based, 8% hysteresis, handoff**

Scheduling granularity. Thermal and heterogeneous-processor studies work with processors, tasks, or neural network subgraphs. LinguaLinked splits models between devices. In contrast, our scheduler acts on conversations assigned to hosts and ensures that there is one active executor per conversation.

Decision factors. The scheduling research considered takes into account the thermal state, processor state, queues, deadlines, and battery life. Our proposal relies on six normalized input factors, although it incorporates hard preconditions. Compatibility and memory allocation requirements are not preferences.

Overhead of decisions. Weighted sum calculation has predictable arithmetic cost and can be reviewed by users. Yet, the probing process, communication with peers, and transmitting context may become the dominant overhead parts. The evaluation should measure all these parts independently rather than report score computation as routing delay.

Stability and adaptability. Continuous monitoring is only useful when it leads to appropriate action. Slight score changes should not trigger frequent switches. Our proposal therefore implements a relative improvement threshold, an adjustable persistence period, and a minimal holding period for voluntary switching. Failures of safety requirements and disconnected hosts have different treatment.

Privacy and trust. Dependence on the cloud is removed but not the trust issue. The chosen peer gets the conversation content. Consequently, consent, authentication, access control, and protected connection become integral parts of our proposal rather than add-ons.

The contribution proposed in this report consists in combining live device scoring, admission guards, stable event-driven host selection, and conversation-level handoffs in a local group of smartphones. The

literature review stimulates this combination but does not exhaustively prove its uniqueness or superiority.

2.3 CONCLUSION

Many references exist for resource-aware scheduling on constrained devices. Thermal studies have been used to argue for live headroom measurement; offloading motivates battery and queue state consideration; distributed inference motivates coordination between devices; and sustained-load benchmarking stresses the importance of evaluating full-length sessions instead of only startup behavior. This design is based on the assumption that each independently executable host corresponds to one conversation. It should be able to cope with the failure of one phone due to its unavailability, overheating, memory issues, or revocation of permission. An easy-to-understand score will be combined with strict admission criteria and a handoff policy to ensure that the adaptive scheme does not degenerate into random switching. The assumptions made in this design, including the one-second heartbeat interval, the sub-80 ms budget for lightweight decisions, and the sub-200 ms routing-control handoff, are all still conjectures at this stage and need to be validated.

CHAPTER 3
PROPOSED SYSTEM

This chapter describes the proposed architecture, the software, hardware and security requirements, the deployment strategy, and the system-flow diagrams.

3.1 SYSTEM ARCHITECTURE

The system consists of the serving environment and the calibration environment. Serving includes discovery, requests, routing, token streaming, event monitoring and handoff. Calibration refreshes device-specific estimates during appropriate idle periods so that benchmarking does not interfere with ongoing conversations. Each phone can play one or multiple logical roles:

- **Origin:** captures the user's request, stores the definitive conversation log, renders the streamed responses, and saves the completed turns.
- **Router:** maintains the current peer telemetry and host assignments, evaluates the eligibility of the candidates, ranks the candidates and coordinates handoffs.
- **Host:** runs the chosen local model, manages context and runtime memory, and streams the output.
- **Member:** participates in discovery and telemetry and can become a host when eligible and authorized by its owner.

This is a logical role distinction and not a hardware role segregation. One phone can serve both origin and host functions. The current design assumes one active routing authority per group and assignment epoch, which denotes routing decisions. Recovery from router loss involves replacement of the authority without creating two simultaneously valid host assignments.

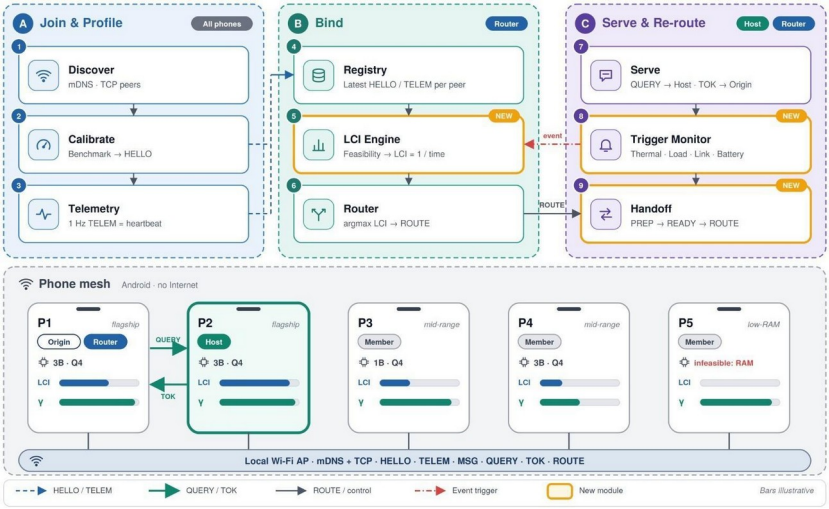


Figure 3.1. System Architecture, Phone Roles over the Local Mesh

Figure 3.1 depicts how the origin, router, host and member roles communicate over the local mesh. Requests and output are part of the serving pipeline. Telemetry and event logs can provide an idle calibration pipeline, but calibration will never require uploading the conversation log.

Serving environment. A telemetry coroutine gathers the supported device-state observations, stamps the snapshot and pushes it to the router. The router analyzes the requested model and the expected

context memory size relative to the latest valid state. It either keeps the eligible host, chooses a new host or produces an explicit unavailable result. A bounded queue avoids accumulation of requests.

Event monitoring and handoff. The monitor distinguishes voluntary optimization from forced recovery. A voluntary switch needs a better candidate. Forced recovery happens in the case of disconnect, withdrawal of consent, critical thermal state or failure to admit memory. The prepare step tests model and context readiness; ready implies admission; route creates a new assignment epoch. Late output from an outdated assignment is not accepted as a second response.

Calibration environment. A short local benchmark assesses the prefill and decode capabilities of the supported runtime and model. Comparison of the actual service-time history and the benchmarks can reveal any drift. A simple regression or correction factor can be fitted locally if enough samples are available. Calibration constants are atomically refreshed during idle periods and versioned to explain to the router which profile it relied on. The telemetry buffer holds the bounded observations and routing events and not a history of the user’s private chats. The conversation transcript remains under origin control unless the user decides on a retention or exporting policy. If there is no available host at all, the system tells the reason and does not try indefinitely.

3.2 SYSTEM REQUIREMENTS

The system is expected to have the following components: an Android client, a compatible native inference runtime, a local communication layer and enough available memory in at least one device to run the chosen model. No cloud warehouse, training server or internet connection is required during the conversation.

3.2.1 SOFTWARE REQUIREMENTS

As the main programming language for the Android application we propose Kotlin. Kotlin provides a consistent client implementation of chat interaction, asynchronous telemetry and local networking capabilities. Inference is done via a C++ runtime such as llama.cpp and accessed through JNI. Specifics of the target platform version and runtime revision will be fixed during implementation and will not change during execution. The native API has methods to load models, tokenize input, create context, generate output, cancel and destroy context. None of these calls may block the UI thread. Model identity, quantization format, tokenizer compatibility and context limits must be saved by the application so the router does not treat two different model files as equal just because they have the same name.

Networking takes place via a framed local protocol over an authenticated transport. Discovery can be done using any supported Android local networking capability or explicitly configured peer addresses. The communication module must account for denial of permissions, lack of discovery support, network change, partial read, timeout and reconnection attempts. Offline functionality requires the

models and the application assets to be pre-installed. The software requirements are listed in Table 3.1.

Table 3.1. Software requirements of the proposed system

Component	Requirement	Purpose
Android platform	Android release supporting required networking and telemetry APIs	Client execution, permissions, telemetry, and lifecycle handling
Kotlin and coroutines	Pinned compiler and compatible coroutine library	Application logic and asynchronous tasks
C++ and Android NDK	Toolchain compatible with selected runtime revision	Native inference and memory management
Inference runtime	llama.cpp	Quantized model loading and generation
JNI interface	Validated native bindings	Communication between Kotlin and C++
Local networking	Supported discovery; framed authenticated connections	Peer registry, telemetry, requests, and responses
Model assets	Matching model hash, tokenizer, quantization, and context limit	Compatible independent host execution
Local storage	Bounded transcript and telemetry storage	Recovery, diagnostics, and evaluation export
Dashboard UI	Android-native interface	Chat interaction and routing explanations
Development tools	Android build tools and device profiling tools	Build, deployment, debugging, and measurement

Python can be utilized independently from the serving application to examine the exported experimental logs. It is not suggested as an Android serving language, and the design does not rely on Anaconda, container orchestration, or a remote server that is running all the time.

3.2.2 HARDWARE REQUIREMENTS

In this design, the deployment is done using a set of heterogeneous Android smartphones connected through a local Wi-Fi network. The suitability of a device is judged according to the selected model and workload, and not according to desktop-like hardware requirements. A smartphone can be used as a client even though it has insufficient memory to be used as an inference host. The host memory must take into account model weights, runtime buffers, context state, operating system requirements, and some margin for safety. Installed RAM is not sufficient to guarantee feasibility since free memory varies depending on the running applications and system activity. In addition, the number of cores cannot be used as a replacement for sustainable generation speed.

Low-, mid-, and high-performance smartphones should be included in the evaluation, if they are available; however, the real identity and specifications of the devices should be reported in the testbed table, and the devices should use the same model and generation configuration. Unavailable sensors are to be treated as such and not as zero temperature, infinite battery, or infinite headroom. An appropriate conservative default and the source of each measurement have to be presented on the dashboard. The system must obey OS-imposed limits instead of working around thermal protections. The hardware requirements are listed in Table 3.2.

Table 3.2. Hardware requirements of the proposed system

Component	Requirement	Purpose
Phone group	At least two compatible phones for handoff; five in the evaluation	Origin, router, host, and alternative host roles
Processor	CPU supported by the native runtime	Local inference and lightweight probes
RAM	Free memory exceeding model, context, buffers, and reserve	Safe admission and OOM avoidance
Storage	Space for model assets and bounded local logs	Offline model availability and diagnostics
Wireless network	Shared local Wi-Fi or supported peer arrangement	Peer communication without required internet access
Battery and power	User-configured participation limit; chargers for controlled tests	Safe participation and endurance experiments
Thermal observations	Available OS thermal status; optional validated sensors	Thermal exclusion and sustained-load evaluation
Development computer	Computer capable of building and deploying the client	Development only; not a production inference server
Power measurement	External meter or documented supported readings	Energy estimates with stated uncertainty

3.2.3 SECURITY REQUIREMENTS

The assets are the content of conversations, identities of the peers, model files, assignment authority, and telemetry integrity. Threats include unauthorized participation in the conversation, eavesdropping, forged telemetry, replayed routing messages, invalid requests, and accidental logging of private data.

Peer admission and consent. A phone has to be authorized explicitly for participation in the conversation. All participants can observe the membership list of the group, enable or disable their hosting

capability, and leave the conversation at any time. The design assumes that not all devices connected to the same access point are trusted.

Transport and authentication. All requests, transcripts, and responses use authenticated encrypted transport. Session IDs, nonces, sequence numbers, and assignment epochs ensure that obsolete control messages are not treated as valid authority. Credentials and pairing material are stored in an appropriate secure local store.

Input and model validation. The application enforces limits on message size, queue length, context length, and request rate. The model hash and tokenizer identifier are validated before handoff. Malformed or unsupported peers are rejected with the reason specified.

Telemetry integrity. The router performs identity, freshness, and sequencing checks on messages. They prevent network spoofing, but do not allow verifying that the authenticated phone reported true sensor measurements. The first version of the system assumes honest participants; attestation and adversary scoring are to be implemented in the future.

Retention and access control. Debugging exports exclude the conversation text by default. Diagnostic logs contain only the minimal set of fields needed for diagnosis. Users have control over transcript retention and deletion. Offline processing does not hide the conversation from the inference host.

3.2.4 DEPLOYMENT AND SCALING

Deployment starts with installation of the Android application and model assets on the target hosts. Participants establish and join a local network, pair devices, and elect a routing authority. The router constructs its own registry using capability announcements and fresh telemetry before accepting any requests. For a small number N of phones, every participant makes one compact state announcement per heartbeat. The router caches the most recent valid announcement and does a linear scan among the eligible candidates. This way, the router avoids the need to broadcast full telemetry history or to re-run an expensive benchmark for every request. The one-second interval is configurable, as it needs to be justified by its cost and utility.

To scale to many conversations, one needs queue management, host admission control, and reserved memory for each context. Host admission criteria cannot be based on the ability to load the model once. The first design is focused on the correctness of a small trusted cluster; multi-router support and large-group scheduling are extensions. Router failure is managed by a well-defined procedure using changing authority epochs. The new router has to reconcile itself with the most recent transcript and valid assignments before serving requests. A network partition might require suspending some conversations to prevent the fiction of a global authority.

3.3 SYSTEM DESIGN

The diagrams below show how control and data move through the system. They are design diagrams, not screenshots from a working application. The activity diagrams include entry and exit points, process steps, decisions and arrows showing the flow. The data flow diagram shows where data goes and where it is stored; it is not a UML activity diagram.

3.3.1 ACTIVITY DIAGRAM

The router activity starts when a request or a routing event comes in. It updates the state, looks at the possible options and checks whether the current setup can stay the same. An optional change needs to follow the improvement rule; a major problem causes a recovery process. If no phone can handle the request, the person who made the request gets a message that it is not possible.

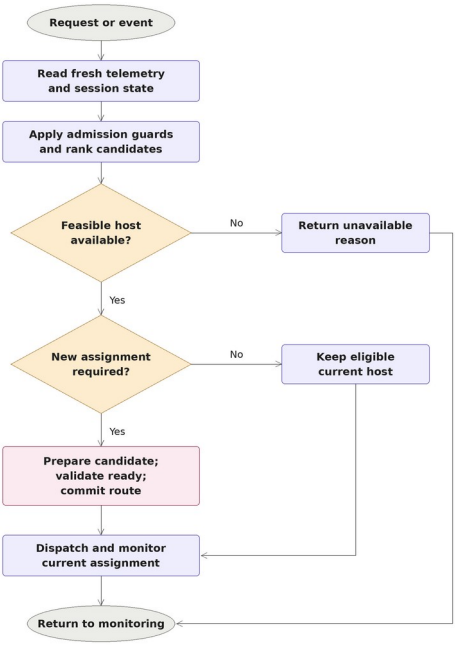


Figure 3.2. Activity Diagram for the Router Environment

Figure 3.2 summarizes the successful preparation path. Preparation failure does not commit a route: the router retains an eligible old host or tries the next feasible candidate within a bounded retry policy. The decision is recorded with its reason and epoch.

The host environment starts with a preparation or serving request. It authenticates the caller, validates model identity, reserves memory, and reconstructs the required context. Only after successful admission does it report readiness. Generation is cancellable and continuously subject to runtime safety checks.

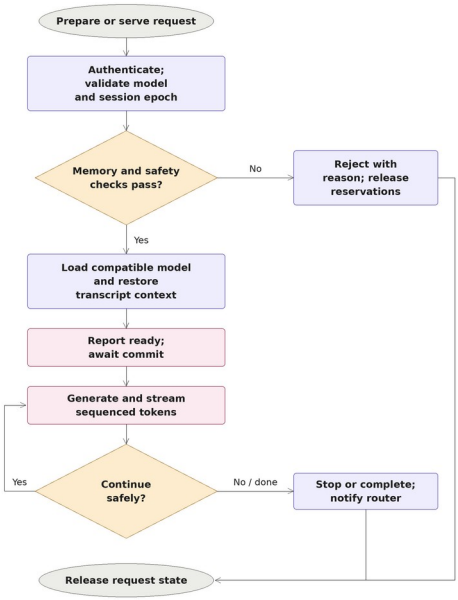


Figure 3.3. Activity Diagram for the Host Environment

Figure 3.3 separates readiness from committed execution. A host must not generate a second authoritative response merely because it has prepared a context. Readiness messages expire, and a failed or cancelled preparation releases reserved memory.

3.3.2 DATA FLOW DIAGRAM

The system has four principal data streams: capability and telemetry messages, requests and transcripts, token responses, and diagnostic events. Conversation data move from the origin to the selected host; device-state data move from participants to the router; diagnostic summaries are stored locally for calibration and evaluation.

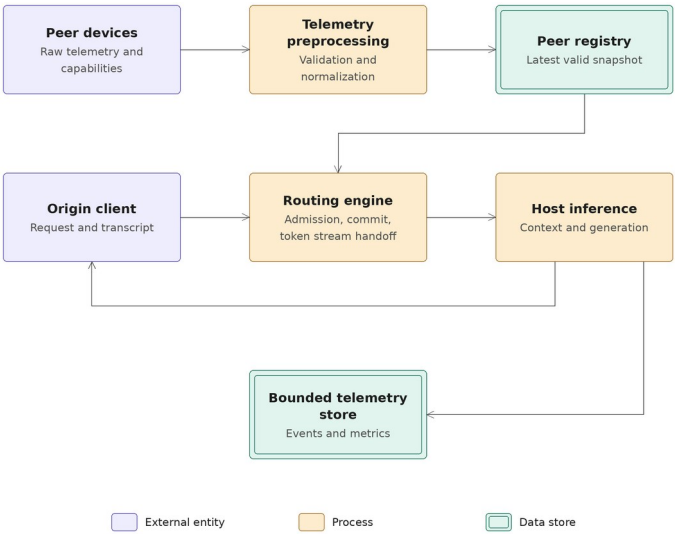


Figure 3.4. Data Flow Diagram of the System

Figure 3.4 is a level-1 logical data flow diagram. Double-bordered stores distinguish persisted state from processing components. The diagram does not imply cloud storage or a training data warehouse. Raw telemetry is transformed into bounded, timestamped observations, while the origin retains the authoritative record of completed conversation turns.

During handoff, the router sends the new host the model identity, assignment information, and authorized conversation context. Context transfer is distinct from ordinary telemetry. The selected host acknowledges readiness, receives the committed route, and streams output with request and token sequence identifiers so that the origin can reject stale or duplicated messages.

CHAPTER 4
PROJECT DESCRIPTION

This chapter describes the technology choices and the five logical modules. The modules are boundaries within the proposed Android application and native runtime; they are not assumed to be Docker containers or remote services.

4.1 TECHNOLOGIES AND METHODOLOGY

The Android client is written in Kotlin; tasks related to telemetry, discovery, network I/O and user interaction utilize coroutines. The native C++ inference engine uses JNI to load the model and perform prefill and decode. The quantized model assets are kept locally to be able to run the session once the connectivity to the outside world is lost. The local communication channel transmits capability advertisements, heartbeats, authenticated control messages, chat requests and streamed tokens. The message format includes protocol version, peer identity, session identifier, assignment epoch, sequence number and payload size. The protocol and model version compatibility are checked before admitting a participant as a potential host.

The dashboard uses a native Android interface. Local bounded storage is used for storing transcripts, calibration constants and diagnostics. The exported evaluation logs can be processed offline, but the data processing toolchain is not part of the server-side dependency tree. The initial implementation avoids distributed training and per-token

model partitioning. The aim of this implementation is to allocate the existing model to a fitting phone and serve it stably despite the changes in device conditions.

4.2 MAIN MODULES OF THE PROJECT

The suggested application consists of five interacting modules:

1. Mesh Communication Module;
2. LCI Computation Module;
3. Event-Based Routing Module;
4. Host Inference Module;
5. Dashboard.

Telemetry preprocessing serves as an input step for both LCI computation and event monitoring. Each of the modules has explicit success and failure outcomes. A failed probe, a disconnected peer and a model mismatch are distinct situations and should not be considered a reason for a low score.

4.2.1 MESH COMMUNICATION MODULE

The mesh communication module locates peers, admits devices, keeps connections alive, and carries application messages. It gives the routing engine a registry showing each node’s identity, willingness to serve, available models, capabilities, last-seen time, and connection state. For discovery and admission, a joining phone advertises its protocol version and serving capabilities. The group may admit or pair with it before accepting requests or private context. Model advertisements list an identifier, content hash, quantization, and

compatibility details. Discovery alone doesn't allow a device to join the conversation. Each participant also sends regular telemetry snapshots. The router records their arrival with its own monotonic clock, then checks the sequence numbers before using the record for routing. A participant that sends a telemetry snapshot older than the configured freshness limit is left out of host admission. Missed heartbeat limits and transport errors are logged separately, since delayed packets do not necessarily mean the device has disappeared.

Framing and streaming use bounded control frames with explicit type and size. Each token stream includes a request ID, assignment epoch, and token sequence number. The origin accepts output only from its committed host assignment. Partial messages, oversized payloads, or protocol mismatches produce errors rather than an undefined state.

On reconnect, the peer uses exponential backoff and advertises its capabilities and available models. It must know the current epoch before it can serve at all. Reconnection doesn't revive an expired assignment or resend the last user request. That is the boundary. The logical peer mesh can run over a local access point, and the prototype should not depend on multi-hop networking or privileged hardware access. Network availability, permissions, topology, and model availability are logged for every evaluation run.

4.2.2 LCI COMPUTATION MODULE

The LCI computation module turns a bunch of different measurements into one suitability score that you can actually compare. It takes six signals: decode speed, memory headroom, thermal headroom, queue

availability, battery state, and network RTT. After normalization, they all point the same way, so a bigger value means a better condition. For a helpful observation x with calibration bounds ℓ and u, the proposed normalization is

(4.1)

For cost observations such as RTT and queue delay, the direction is flipped.

The annoying part is the bounds. They need to be declared and versioned, not quietly recalculated from whatever peers are nearby. Otherwise, an 8% score gain could mean something different just because a new phone shows up in the group. Let d_i, m_i, h_i, q_i, b_i, and r_i denote the normalized decode, memory, thermal, queue, battery, and network signals for phone i. A proposed weighted score is

(4.2)

where every weight is nonnegative and

(4.3)

The supplied pipeline uses the weight vector $(w_d, w_m, w_h, w_q, w_b, w_r) = (0.30, 0.22, 0.18, 0.12, 0.10, 0.08)$. Its displayed index is $I_i = 1000\ \mathrm{LCI}_i$. The signals and their weights are summarized in Table 4.1.

Table 4.1. LCI signals, orientation and weights

Signal	Symbol	Orientation	Weight
Decode speed	d_i	Benefit (higher is better)	0.30
Memory headroom	m_i	Benefit (higher is better)	0.22
Thermal headroom	h_i	Benefit (higher is better)	0.18
Queue availability	q_i	Cost reversed (queue delay)	0.12
Battery state	b_i	Benefit (higher is better)	0.10
Network RTT	r_i	Cost reversed (RTT)	0.08
Total			**1.00**

The same telemetry can be checked against every available model, so model fit gets judged along with device fit.

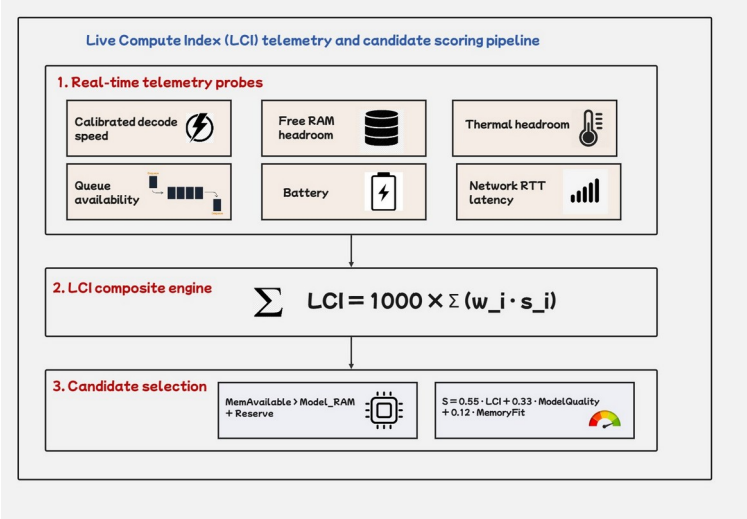


Figure 4.1. LCI Telemetry and Candidate Scoring Pipeline

Figure 4.1 keeps feasibility separate from ranking, which is the sensible part. A phone-model pair only counts as feasible if it is reachable, trusted, consented, fresh enough, compatible with the

requested model, and inside the configured resource limits. Free memory has to satisfy

(4.4)

That context estimate changes with the model setup and the requested length. The reserve is set conservatively, and it does not pretend all free memory is really usable.

Once a candidate passes that test, the pipeline mixes the displayed LCI, model quality Q_i, and memory fit F_i on a common [0, 1000] scale. The score is

(4.5)

where F is the feasible set. A candidate gets cut before scoring if the model does not fit or memory is too low. Model quality is judged alongside the device's current condition when available models are compared. Short-term smoothing can help with sensor noise, but it should not hide a thermal or memory warning. Stale or missing measurements lower confidence and may force a conservative exclusion. The dashboard shows raw observations, normalized terms, weight contributions, model availability, and guard outcomes, so each LCI value stays readable during routing.

4.2.3 EVENT-BASED ROUTING MODULE

This module decides when an assignment should be reconsidered. It looks at host status, device and model scores, network events, request outcomes, and whether the user is still willing to use certain devices. A score changing by itself does not move the session.

Elective triggers include longer queue delays, thermal headroom falling below a non-critical level, a configured battery participation threshold, a stronger device-model candidate, or a query that fits another available model better. A persistence window and minimum residence time help filter out temporary changes. This filtering avoids needless session moves.

Some events force migration. These include disconnection, a critical safety condition, failed memory admission, inference failure, model unavailability, or the user withdrawing hosting consent. They make the current assignment invalid and override its normal stickiness. If no safe device or suitable model is available, the system pauses the conversation and returns a recoverable error. The two classes of trigger are compared in Table 4.2.

Table 4.2. Elective and forced routing triggers

Class	Triggers	Handling
Elective	Longer queue delay; thermal headroom below a non-critical level; battery participation threshold; stronger device-model candidate; query that fits another model better	Considered only after the persistence window and minimum residence time, and only if $S_j > (1 + \delta) S_h$
Forced	Disconnection; critical safety condition; failed memory admission; inference failure; model unavailability; hosting consent withdrawn	Current assignment becomes invalid; stickiness is overridden; forced recovery, or a recoverable pause if no safe device or model exists

For current assignment *h* and candidate *j*, the suggested relative stickiness rule is

(4.6)

It means an 8% relative bump, not just adding 0.08 to a normalized score. When the score is zero, elective migration needs a real absolute gain before it can happen. A model switch on the same host and a host handoff are still treated as different actions, since loading the model and rebuilding context can add delay. If feasibility is lost, the system always goes down the forced recovery path.

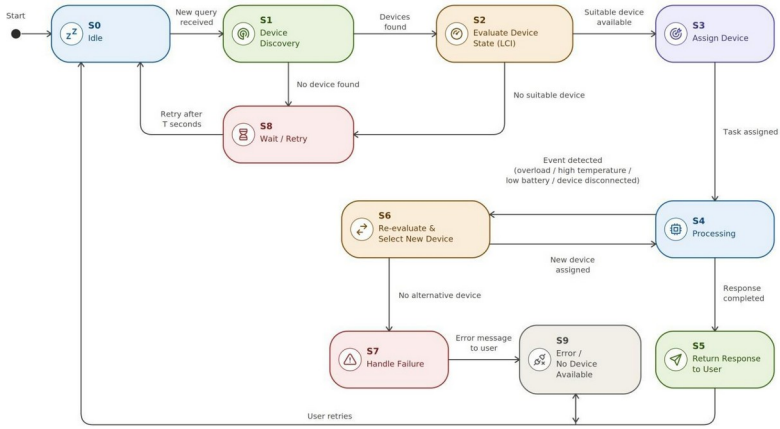


Figure 4.2. Event-Based Routing State Machine

Figure 4.2 moves through discovery, scoring, assignment, processing, output delivery, and then re-evaluation. The retry and no-device branches separate plain unavailability from a turn that actually finished. The prepare-ready-route protocol only commits a route once readiness is confirmed (Table 4.3). The prepare message carries the session identifier, model name, context digest, and proposed epoch, and the candidate checks compatibility before reserving resources. Ready confirms the admitted state and when it expires. Route locks in

the epoch, and after that the origin only takes output from that assignment. Timers clear out abandoned prep state.

Table 4.3. Prepare–ready–route handoff messages

Message	Carries	Effect
Prepare	Session identifier, model name, context digest, proposed epoch	Candidate checks compatibility and reserves resources
Ready	Admitted state and expiry time	Confirms readiness; reservation expires if unused
Route	Committed assignment epoch	Locks in the epoch; origin accepts output only from this assignment

For elective migration, the cleanest move is a turn boundary. If the old host dies mid-generation, the origin keeps the finished turns and treats the unfinished response as interrupted. It can send that incomplete request to the new host, though exact bit-for-bit continuity is not guaranteed.

4.2.4 HOST INFERENCE MODULE

The host inference module runs the chosen model and gives a cancellable request interface. Its job is checking the model, reserving memory, building the context, doing prefill, decoding, streaming tokens, and then cleaning up. Native resources are released when the task ends, gets canceled, times out, or fails.

Model readiness. The host checks the model hash, tokenizer identity, runtime settings, and the allowed context length. If the selected model is already in memory, setup is faster. When a switch is needed, the current model may have to be unloaded before the new one is loaded,

depending on memory. The router has to think about loading and initialization cost before it switches.

Conversation reconstruction. The origin sends the transcript and prompt template. The host reconstructs the context on its own. This preserves the conversation flow, although prefill may take a little longer. If the model changes, rebuilding the context and running prefill again count toward switching cost, not ordinary inference latency.

Streaming. Output is streamed in ordered chunks marked with request and epoch IDs. Backpressure prevents clients from endlessly filling buffers, while cancellation reaches the native generation loop. After cancellation, output cannot resume once a newer epoch has been accepted.

Resource protection. Resource checks happen before generation and continue during execution as needed. Queue size and context count are both limited. If there is thermal stress or memory pressure, the system cancels or blocks new requests depending on the policy.

Failure reporting. Failure reporting is split pretty carefully: model load problems, not enough memory, invalid context, connection loss, cancellation, runtime inference errors. It sends events to the router and short summaries to the dashboard.

Model loading, initialization, and context reconstruction delays are logged separately, which matters if you want to see where switching overhead really comes from. The module can start with CPU inference

and add compatible accelerator backends later. Hardware acceleration needs testing on actual devices, not trust in the SoC spec sheet. Runtime, model, and backend versions stay part of the experiment setup.

4.2.5 DASHBOARD

The dashboard brings together the group chat and a clear view of routing state. It shows connected peers, logical role, hosting permission, available models, telemetry data, normalized score contribution, and eligibility. That matters because model availability and device limits are shown separately, so a user can tell the difference between an unavailable model and one that just scored poorly. Unavailable sensor data is marked plainly instead of being treated like real readings.

A discussion area shows the selected host, selected model, assignment epoch, request status, and the last routing decision. Participants can see whether the system stayed with the current phone and model, switched models, or handed the session to another host because of stickiness, memory admission, model availability, disconnect, or thermal events. The point is simple: a high score does not promise good performance.

Control options let users enable or disable hosting, set battery participation level, leave the group, delete retained transcripts, and export sanitized diagnostic logs. The interface keeps to conservative settings and does not expose all scoring weights.

CHAPTER 5
IMPLEMENTATION AND RESULTS

This chapter presents the workflow and the evaluation of the Phase I implementation through diagrams, plots and tables. Values are those reported by the Phase I evaluation runs and are reproduced as reported, not generated or estimated. The five-phone comparison uses Qwen 0.6B Q8_0, a 2048-token context, four CPU threads, and llama.cpp. The performance runs report 50 runs per configuration with 150 output tokens; the prompt-length runs report 20 repetitions per cell.

5.1 IMPLEMENTATION RESULTS

Telemetry preprocessing gives validated input to the LCI engine and the event monitor. A device snapshot consists of peer identity, sequence number, model inventory version, raw telemetry data, source information, validation indicators, and timestamps. The router records its reception time through its own monotonic clock to measure freshness without the need for synchronized phone wall clocks.

5.1.1 TELEMETRY COLLECTION AND PREPROCESSING

The collector gathers memory usage statistics, thermal status support, battery level and charge state, current queue state, decode throughput observation, and network RTT. Observed inference-speed telemetry is used if valid, recently submitted request telemetry exists; otherwise, a bounded benchmark estimate may be used instead, with its age and provenance noted. Each observation undergoes range checking and

missing-value processing. It is not assumed that the thermal measurement from different devices represents the same physical component: the OS thermal status and a validated temperature sensor reading are different fields. Missing values do not create unfairly positive scores. The preprocessing step matches observations into snapshots, applies normalization bounds as specified, and may smooth noncritical variations. Admission decisions receive unsmoothed critical signals where appropriate. Decode measurements differentiate prefilling and token generation, and network RTT is not used to estimate inference service time.

The evaluation uses five devices, listed in Table 5.1. The system keeps track of device identity and processor specifications and does not infer installed memory or operating system versions through capability tiers. Controlled comparisons use matching model assets and account for variation in initial conditions.

Table 5.1. Device configuration in the five-phone evaluation

Phone	Identifier	Processor
Realme GT 6T	RMX3853	Snapdragon 7+ Gen 3
Samsung Galaxy S23 FE	SM-S711B	Exynos 2200
Google Pixel 7 family	Pixel 7 / 7 Pro	Tensor G2
Realme 11 Pro+ 5G	RMX3741	Dimensity 7050
Realme 5 Pro	RMX1971	Snapdragon 712

The Pixel entry is identified as Pixel 7 / Pixel 7 Pro in the evaluation record; it is counted as one mesh participant.

Figure 5.1 compares the initial and sustained decode speed along with charge-counter energy estimates. The evaluation reports 182.4, 458.0, 426.5, 314.2, and 712.8 mJ/token for the GT 6T, S23 FE, Pixel 7,

Realme 11 Pro+, and Realme 5 Pro, respectively. Charge-to-energy conversion requires voltage and an accounting interval; these estimates are reproduced from the figure and were not independently calibrated.

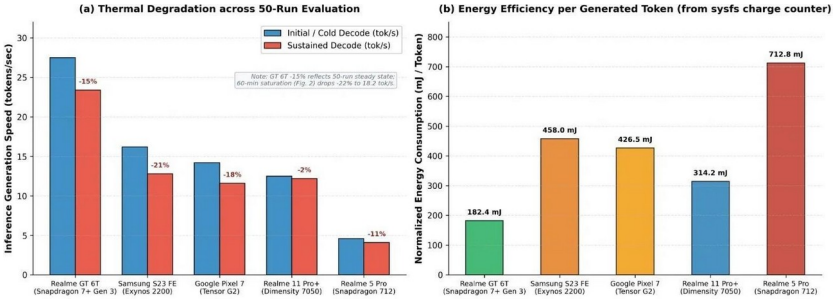


Figure 5.1. Thermal Degradation and Energy per Generated Token

The collector is assessed for overhead, measurement freshness, missing-data frequency, and interference with generation. The proposed sub-80 ms budget applies to the stated lightweight collection and scoring path. It does not include full benchmarking, model loading, or context reconstruction.

5.1.2 IMPLEMENTATION OF LCI COMPUTATION

The LCI evaluation runs Equations 4.1 to 4.5 against the versioned snapshot. The order of operations is: validate observations, check hard admission guards, normalize supported observations, compute the weighted LCI, enforce the readiness policy, and rank the candidates. A stable peer ID can break ties without causing host hopping. The scoring function yields both a scalar and an explanation of it. The explanation consists of raw inputs, normalized terms, weight,

confidence bit, exclusion criteria, readiness of the model, and calibration version. This facilitates debugging and ensures that the dashboard can explain a result even when independent re-computation would give a different number.

Checks for implementation correctness need to test normalization bounds, cost metric reversals, invalid bound intervals, unsupported sensors, outdated snapshots, insufficient memory, incompatible models, and ties. A high LCI may not be used to bypass the feasibility set. Weight sensitivity and ablation tests need to compare the full score against scores with individual signals dropped.

Evaluation metrics. Routing overhead is the elapsed time between request receipt by the routing authority and the dispatch decision. TTFT is calculated as the elapsed time between request submission and the first acceptable output token. Decode throughput does not account for prefill time and is calculated as

(5.1)

The energy per token metric is calculated as

(5.2)

The measurement interval and accounting boundary are to be specified explicitly; for example, total session energy and the energy of inference are different quantities. Request success is defined prior to the experiment as a completed request, with the right assignment and meeting any deadline requirements. The request success rate is reported as

(5.3)

Requests canceled, requests rejected, retried requests and partial responses are counted explicitly rather than filtered out of the denominator.

Table 5.2. Inference throughput and request latency

Configuration	Prefill (tok/s)	Decode (tok/s)	TTFT 64 (ms)	TTFT 256 (ms)	150-token latency (s)
Event-based mesh	63.5	22.8	1076	4144	7.62
Static mesh (GT 6T)	58.2	19.8	1174	4695	8.75
Round-robin mesh	24.4	12.4	2755	11120	18.65
GT 6T standalone	64.2	23.4	1045	4230	7.95
S23 FE standalone	36.5	12.8	1838	7378	13.50
Pixel 7 standalone	34.8	11.6	1930	7733	14.80
11 Pro+ standalone	26.8	12.2	2476	9859	14.70
5 Pro standalone	10.4	4.1	6405	26374	42.80

Qwen 0.6B Q8_0, 2048-token context, four CPU threads. TTFT 64 and TTFT 256 denote prompt lengths. Values are retained as reported rather than recomputed from separately aggregated throughput.

Table 5.3. Tail latency, thermal metric and request success

Configuration	p95 latency (s)	Thermal metric (%)	Success (%)
Event-based mesh	8.45	3.2	99.4
Static mesh (GT 6T)	11.60	34.0	96.0
Round-robin mesh	44.20	14.8	82.0
GT 6T standalone	9.85	26.5	98.0
S23 FE standalone	28.50	42.0	92.0
Pixel 7 standalone	16.80	18.5	98.0
11 Pro+ standalone	16.10	24.0	97.5
5 Pro standalone	52.40	65.0	60.0

The thermal metric reproduces the “Thermal Throttling” column of the evaluation record; its aggregation is not specified. The p95 values measure request turnaround,

not routing-control overhead. Fractional success rates are retained without inventing request counts.

Tables 5.2 and 5.3 present the performance results in readable form. Static mesh assignment uses a fixed GT 6T host; round robin cycles over the five participants. Standalone rows describe individual devices. The sustained-service benefit of the adaptive policy must be distinguished from the cold dispatch cost.

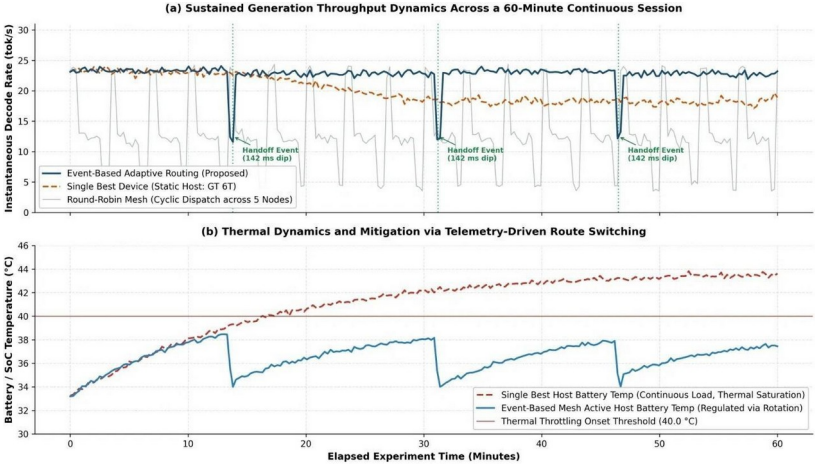


Figure 5.2. Sustained Throughput and Battery-Temperature Dynamics during the Continuous Session

Figure 5.2 shows stable adaptive throughput, declining fixed-host performance, and variable round-robin service. Marked handoffs coincide with reductions in the adaptive host's battery-temperature trace. Battery temperature is not relabeled as processor temperature; the summaries do not establish behavior outside this workload.

5.1.3 IMPLEMENTATION OF EVENT-BASED ROUTING

The event router keeps explicit session states and ensures a monotonically increasing epoch commitment of host assignments. It recognizes forced changes and elective improvements, and serializes all handoffs so that no multiple monitors independently commit a session on different hosts. The elective change process validates the eligibility of the current host, the 8% elective margin, the persistence period, and the minimum dwell time. It prepares a suitable candidate while retaining the previous assignment, which continues to hold the session until the candidate is ready and the new route is committed. Reservations are released if the preparation failed or expired.

Forced recovery happens once the current assignment is invalid. The router selects a suitable candidate without applying the elective margin. If no such candidate exists, the origin retains completed turns, shows the interruption, and waits for a valid retry or a change of participants. Sending a prepare message alone is not counted as a successful handoff. For measurement, the handoff time is decomposed into

(5.4)

The goal of sub-200 ms handoff is limited to the warm-host routing-control path. The total interruption is measured separately for cold and warm routes, including context preparation and generation restart times.

Table 5.4. Time to first token across prompt lengths

Configuration	64 tokens (ms)	128 tokens (ms)	256 tokens (ms)	512 tokens (ms)
Event-based mesh	1076	2095	4144	8435
Static mesh (GT 6T)	1174	2330	4695	9902
Round-robin mesh	2755	5510	11120	23835
GT 6T standalone	1045	2085	4230	8875
S23 FE standalone	1838	3660	7378	15506
Pixel 7 standalone	1930	3845	7733	16192
11 Pro+ standalone	2476	4920	9859	20986
5 Pro standalone	6405	12925	26374	57780

20 repetitions per cell. TTFT includes prefill, first-token decode, and dispatch overhead; it is not a routing-decision timing.

Table 5.5. Prefill throughput across prompt lengths

Configuration	64 tokens (tok/s)	256 tokens (tok/s)	512 tokens (tok/s)
Event-based mesh	63.5	62.8	61.2
Static mesh (GT 6T)	58.2	55.4	52.1
Round-robin mesh	24.4	23.3	21.6
GT 6T standalone	64.2	61.2	58.0
S23 FE standalone	36.5	35.1	33.2
Pixel 7 standalone	34.8	33.5	31.8
11 Pro+ standalone	26.8	26.2	24.5
5 Pro standalone	10.4	9.8	8.9

Qwen 0.6B Q8_0. Round-robin throughput is reported as an effective rate across the rotating nodes.

Tables 5.4 and 5.5 distinguish prompt-dependent TTFT from prefill throughput. At 512 tokens, adaptive TTFT is 8435 ms, versus 9902 ms for static assignment and 23835 ms for round robin. The evaluation associates the Realme 5 Pro’s 57780 ms result with memory pressure near its stated 4 GB limit, reinforcing context-dependent memory admission.

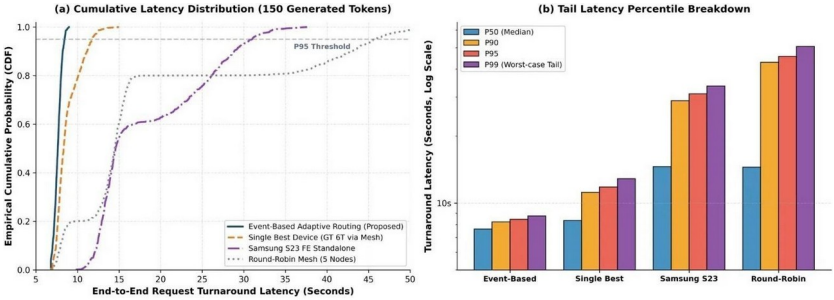


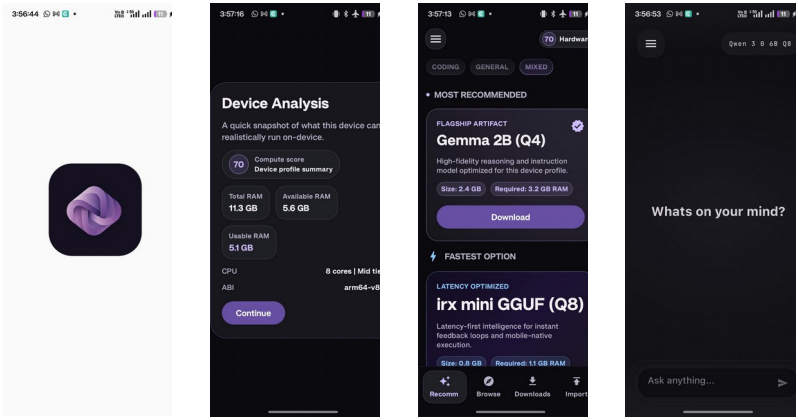
Figure 5.3. Request-Latency Cumulative Distribution and Tail Percentiles

Fault injection should cover host disconnection, stale telemetry, memory admission failure, critical thermal events, readiness timeout, duplicated control messages, delayed old-host tokens, and router replacement. Each test checks the visible outcome, the retained transcript, the committed epoch, the cleanup behavior, and whether the system generates more than one authoritative response. Synthetic events can validate control logic, but they do not substitute for measured physical thermal behavior or power consumption. A simulated thermal event is labeled as such. Similarly, one short uninterrupted run cannot establish a population-level MTBF or a long-term battery-health benefit.

5.1.4 DASHBOARD AND APPLICATION SCREENSHOTS

The implemented dashboard shows the active session data, host identity, peer eligibility status, current score data and the explanation behind the last routing decision. It displays heartbeat age and sensor availability so that users do not misinterpret a lost reading as a normal resource state. The interface shows the states of candidate preparation,

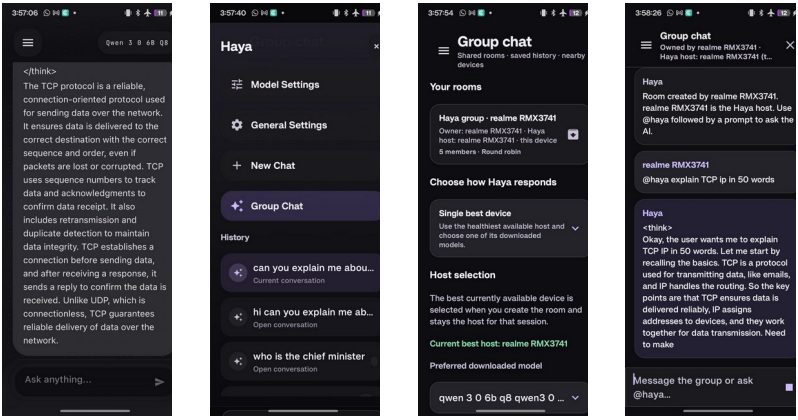
context rebuilding, response streaming, current host retention, failure recovery, and waiting for a host to become available. Users can disable hosting or withdraw from the group without requiring access to internal router settings. The diagnostic view presents host selection, admission outcomes, handoff events, and recovery. Figures 5.4 and 5.5 show screens of the HAYA application.



(a) Application launch screen (b) Device analysis (c) Model recommendations (d) Individual chat interface

Figure 5.4. HAYA Application Access and Device Configuration

The launch screen (a) displays the HAYA application logo when the app is opened. The device analysis screen (b) presents the compute score, total and available RAM, usable RAM, CPU core count, and device ABI. The recommendation screen (c) lists model options with their download sizes and RAM requirements alongside task-category filters. The individual chat screen (d) displays the selected model and provides an input field for entering prompts.



(a) Individual chat response (b) Navigation and conversation history (c) Group configuration and host selection (d) Group chat interaction

Figure 5.5. HAYA Chat Output and Group Interaction

The conversation view (a) displays a model-generated explanation of the TCP protocol. The navigation menu (b) provides access to model settings, general settings, new chats, group chat, and previous conversations. The group configuration screen (c) displays a saved room, response-mode options, the current best host, and a preferred downloaded model selector. The shared conversation (d) displays the room host, a participant's @haya prompt, and the model output being generated.

5.2 EVALUATION AND RESULT DISCUSSION

The endurance and reliability results of the 60-minute workload are summarized in Tables 5.6 and 5.7. The 28-minute run ends in a timeout and the 14-minute run in a low-memory failure.

Table 5.6. Endurance, battery temperature and failure outcomes

Configuration	Duration (min)	Peak (°C)	Mean (°C)	Reported outcome
Event-based mesh	60	38.4	35.6	Normal; thermally balanced
Static mesh (GT 6T)	60	43.8	41.2	Throttling; decode decrease of 22%
Round-robin mesh	60	37.8	34.6	Frequent timeouts on low-tier node
S23 FE standalone	28	43.8	41.8	Timeout; CPU reported at 400 MHz
5 Pro standalone	14	38.6	36.8	Low-memory termination; available memory below 180 MB

Temperatures follow the battery-temperature interpretation used for Figure 5.2; they are not SoC-temperature measurements.

Table 5.7. Handoff outcomes and availability statistics

Configuration	Handoff (ms)	Handoff success (%)	Reported MTBF (h)
Event-based mesh	142	100.0	> 24.0
Static mesh (GT 6T)	N/A	N/A	2.5
Round-robin mesh	N/A	N/A	0.9
S23 FE standalone	N/A	N/A	0.45
5 Pro standalone	N/A	N/A	0.22

N/A denotes a policy without the adaptive handoff metric, not an empty result. The evaluation labels its last column "Availability MTBF" without observation exposure or failure counts; these are reported statistics, not independently established MTBF estimates. A single 60-minute run cannot establish a > 24-hour MTBF. The 142 ms handoff value does not separately establish cold-loading or full-context reconstruction time.

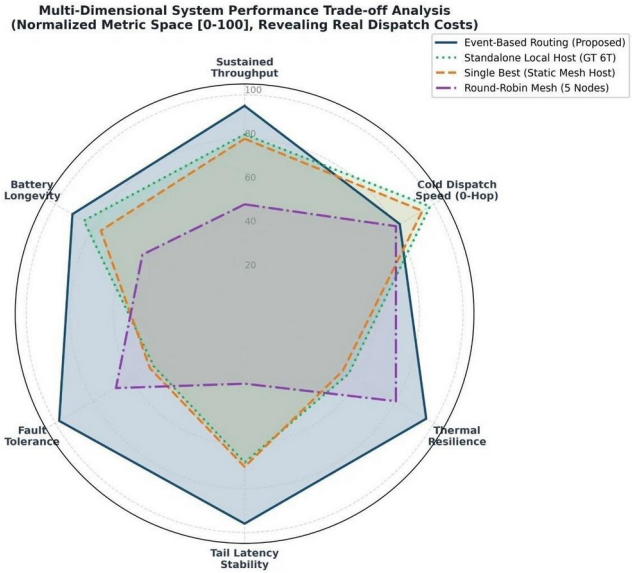


Figure 5.6. Multi-Dimensional Performance Trade-off Analysis

The radar plot is a supplementary summary, not a replacement for raw units and uncertainty. Throughput, latency, and energy measure different quantities; their normalization must not obscure a poor result on a critical safety or reliability axis.

Result discussion. The adaptive mesh reports 22.8 tokens/s sustained decode, compared with 19.8 for static assignment and 12.4 for round robin. Its p95 request latency is 8.45 s versus 11.60 s and 44.20 s. At 512 tokens its TTFT of 8435 ms compares with 9902 ms and 23835 ms. Standalone GT 6T remains faster for 64-token TTFT (1045 versus 1076 ms), showing a dispatch-cost trade-off rather than superiority on every metric. In the endurance run the adaptive policy keeps a peak

battery temperature of 38.4 °C against 43.8 °C for the static host, which throttles with a 22% decode decrease.

The summaries support a workload-specific comparison. Raw logs, exact request denominators, energy conversion details, and long-exposure reliability records are not reconstructed from the figures; no unreported measurements are invented.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This report presents a complete design for device-aware routing of offline group AI chat among the nodes in a local network of smartphones. It poses the problem of choosing a sustainable host among heterogeneous devices and argues that static assignment, random rotation, and single-signal scheduling might not be sufficient for a sustained session. The proposed solution involves a transparent LCI, model compatibility, memory admission, fresh telemetry, user consent, and thermal and battery constraints. The design comprises five modules for communication, scoring, event-based routing, local inference, and explanation. Serving is done separately from idle calibration, so as to avoid competing with the requests while profiling.

The event-based policy helps distinguish optimization from recovery. The designed relative 8% stickiness threshold discourages switching even if the improvement in scores is marginal. In addition, the prepare–ready–route protocol controls the assignment changes. The conversation turn remains under origin control, while incomplete partial output is managed explicitly without the appearance of state transfer. The system uses compatible models locally and depends on the local network being trusted, but recognizes that offline operation does not ensure privacy, telemetry integrity, or bounded handoff time.

The Phase I performance evaluations show a sustained decode speed of 22.8 tokens/s, a p95 request latency of 8.45 s, and 99.4% request success for adaptive routing. The endurance evaluation shows a peak battery temperature of 38.4 °C and a handoff latency of 142 ms. These results make adaptive selection the choice for sustained service in the reported workload; the evaluation summaries do not prove anything else.

6.2 FUTURE WORK

Any future development will require prioritization of implementation and control of its evaluation before introducing more complex scheduling capabilities. The directions for further development include:

1. **Reproducibility:** Store runtime updates, device list, cleaned request logs, number of events, and measurement scripts to allow independent reproduction of the results.
2. **Calibration of weights and thresholds:** Optimize normalization range, LCI weights, freshness, persistence window and dwell time values using independent workloads rather than the same runs as evidence of improvement.
3. **Optimization of context transfer:** Assess compatible context caching, transcript incremental synchronization, and warm standby hosts while taking into account memory and battery overhead.

4. **Concurrent sessions:** Introduce memory reservations per context, fair queuing, participation budget per user and scheduling under concurrent requests.
5. **Router reliability:** Test authority update, epoch consistency, reconnection, and correctness during network partitions.
6. **Device compatibility:** Verify new Android devices, accelerator backends, model sizes and telemetry constraints specific to each platform.
7. **Advanced trust mechanisms:** Investigate ways to detect conflicting peer reports and improve integrity guarantees despite the telemetry being done using software alone.
8. **Health and energy studies for the long term:** Run longer experiments and perform actual power measurements before making any assumptions about battery degradation and longevity.

These enhancements should maintain focus on the essential aspects of the project: transparency of decisions, safety of admission, user consent, and reliable offline service. The enhancements will be accepted only based on measurable performance benefits rather than presumed gains from added complexity.

REFERENCES

[1] T. Tan and G. Cao, "Thermal-Aware Scheduling for Deep Learning on Mobile Devices with NPU," IEEE Transactions on Mobile Computing, vol. 23, no. 12, pp. 10706–10719, 2024, doi: 10.1109/TMC.2024.3379501.

[2] X. Gao, "TAS: A Temperature-Aware Scheduling for Heterogeneous Computing," IEEE Access, vol. 11, pp. 54773–54781, 2023, doi: 10.1109/ACCESS.2023.3281839.

[3] J. Zhang, Y. Wang, T. Huang, F. Dong, W. Zhao, and D. Shen, "Thermal-Aware on-Device Inference Using Single-Layer Parallelization with Heterogeneous Processors," Tsinghua Science and Technology, vol. 28, no. 1, pp. 82–92, 2023, doi: 10.26599/TST.2021.9010075.

[4] Y. Gao, Z. Zhang, P. K. Donta, C. K. Dehury, X. Wang, D. Niyato, and Q. Zhang, "Optimizing Multi-DNN Inference on Mobile Devices through Heterogeneous Processor Co-Execution," arXiv:2503.21109, 2025.

[5] P. Tummalapalli, S. Arayakandy, R. Pal, and K. Kundan, "LLM Inference at the Edge: Mobile, NPU, and GPU Performance Efficiency Trade-offs Under Sustained Load," arXiv:2603.23640, version 2, 2026.

[6] Z. Chen, B. Zhu, J. Wang, H. Shin, A. Nallanathan, and D. Niyato, "Network Edge Inference for Large Language Models: Principles, Techniques, and Opportunities," arXiv:2604.22906, 2026.

[7] J. Zhao, Y. Song, S. Liu, I. G. Harris, and S. Abdu Jyothi, "LinguaLinked: Distributed Large Language Model Inference on Mobile Devices," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics,

Volume 3: System Demonstrations, pp. 160–171, 2024, doi: 10.18653/v1/2024.acl-demos.16.

[8] D. Lim, W. Lee, W.-T. Kim, and I. Joe, "DRL-OS: A Deep Reinforcement Learning-Based Offloading Scheduler in Mobile Edge Computing," Sensors, vol. 22, no. 23, article 9212, 2022, doi: 10.3390/s22239212.

[9] Y. Tang, H. Huang, W. Zhan, G. Min, Z. Duan, and Y. Lei, "Battery-Care Resource Allocation and Task Offloading in Multi-Agent Post-Disaster MEC Environment," arXiv:2312.15380, 2023.

[10] P. Lin, "Fuzzy based priority aware task scheduling optimization for mobile edge computing environments," Scientific Reports, vol. 15, article 39998, 2025, doi: 10.1038/s41598-025-23690-9.

[11] Mesh-LLM contributors, mesh-llm: Distributed AI/LLM for the people, project repository and documentation, accessed October 1, 2026. Repository identifier: Mesh-LLM/mesh-llm on GitHub. [Online]. Available: https://github.com/Mesh-LLM/mesh-llm.

[12] G. Qu, Q. Chen, W. Wei, Z. Lin, X. Chen, and K. Huang, "Mobile Edge Intelligence for Large Language Models: A Contemporary Survey," IEEE Communications Surveys & Tutorials, vol. 27, no. 6, pp. 3820–3860, Dec. 2025, doi: 10.1109/COMST.2025.3527641.

[13] J. Xiao, Q. Huang, X. Chen, and C. Tian, "Large Language Model Performance Benchmarking on Mobile Platforms: A Thorough Evaluation," arXiv preprint arXiv:2410.03613v1, 2024.

[14] H. Chu, J. Fan, J. Deng, B. Yu, L. Xie, L. Liu, H. Ma, and X. Zheng, "Act Before It’s Too Late: Power-Efficient LLM Inference on Mobile Device," in Proc. 24th Annu. Int. Conf.

Mobile Systems, Applications and Services (MobiSys), 2026, pp. 338–351, doi: 10.1145/3745756.3809208.

[15] Z. Xue, Y. Song, Z. Mi, X. Zheng, Y. Xia, and H. Chen, "PowerInfer-2: Fast Large Language Model Inference on a Smartphone," arXiv preprint arXiv:2406.06282, 2024.

[16] Z. Liu, C. Zhao, F. Iandola, C. Lai, Y. Tian, I. Fedorov, Y. Xiong, E. Chang, Y. Shi, R. Krishnamoorthi, L. Lai, and V. Chandra, "MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases," in Proc. 41st Int. Conf. Machine Learning (ICML), Proceedings of Machine Learning Research (PMLR), vol. 235, 2024, pp. 32431–32454.

[17] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU," in Proc. ACM SIGOPS 30th Symp. Operating Systems Principles (SOSP), 2024, pp. 590–606, doi: 10.1145/3694715.3695964.

[18] X. Yan and Y. Ding, "Are We There Yet? A Measurement Study of Efficiency for LLM Applications on Mobile Devices," in Proc. 2nd Int. Workshop on Foundation Models for Cyber-Physical Systems & Internet of Things (FMSys), 2025, pp. 19–24, doi: 10.1145/3722565.3727192.

[19] Y. Yang, Y. Pan, J. Zhang, and W. Ni, "Task Offloading for Collaborative Inference of LLM Agents at the Network Edge," in Proc. IEEE 22nd Int. Conf. Mobile Ad-Hoc and Smart Systems (MASS), Chicago, IL, USA, Oct. 2025, pp. 525–530, doi: 10.1109/MASS66014.2025.00085.

[20] X. Li, H. Li, C. Sun, Q. Fan, Z. Han, and V. C. M. Leung, "Edge-Enhanced Intelligence: A Comprehensive Survey of Large Language Models and Edge-Cloud Computing Synergy," IEEE Communications Surveys & Tutorials, vol.

28, pp. 1248–1284, 2026, doi: 10.1109/COMST.2025.3587225.

[21] M. Zhang, X. Shen, J. Cao, Z. Cui, and S. Jiang, "EdgeShard: Efficient LLM Inference via Collaborative Edge Computing," IEEE Internet of Things Journal, vol. 12, no. 10, pp. 13119–13131, May 2025, doi: 10.1109/JIOT.2024.3524255.

[22] S. Habibi and Ö. Erçetin, "Edge-LLM Inference With Cost-Aware Layer Allocation and Adaptive Scheduling," IEEE Access, vol. 13, pp. 131614–131637, 2025, doi: 10.1109/ACCESS.2025.3592308.

[23] A. Khoshsirat, G. Perin, and M. Rossi, "Decentralized LLM Inference over Edge Networks with Energy Harvesting," in Proc. IEEE Global Communications Conf. (GLOBECOM), Cape Town, South Africa, Dec. 2024, pp. 3703–3708, doi: 10.1109/GLOBECOM52923.2024.10901542.

[24] Y. Chen, Y. Yan, S. Ge, Y. Qin, Y. Zheng, Q. Yang, S. He, Z. Shi, J. Chen, and Y. Shu, "Confidant: Customizing Transformer-based LLMs via Collaborative Training on Mobile Devices," in Proc. 31st Annu. Int. Conf. Mobile Computing and Networking (MobiCom), 2025, pp. 483–497, doi: 10.1145/3680207.3723487.

[25] I. Yaqoob, E. Ahmed, A. Gani, S. Mokhtar, M. Imran, and S. Guizani, "Mobile ad hoc cloud: A survey," Wireless Communications and Mobile Computing, vol. 16, no. 16, pp. 2572–2589, 2016, doi: 10.1002/wcm.2709.

[26] K. Habak, M. H. Ammar, K. A. Harras, and E. W. Zegura, "Femto Clouds: Leveraging Mobile Devices to Provide Cloud Service at the Edge," in Proc. IEEE 8th Int. Conf. Cloud Computing (CLOUD), 2015, pp. 9–16, doi: 10.1109/CLOUD.2015.12.

[27] A. Mtibaa, A. Fahim, K. A. Harras, and M. H. Ammar, "Towards Resource Sharing in Mobile Device Clouds: Power Balancing Across Mobile Devices," in Proc. 2nd ACM SIGCOMM Workshop on Mobile Cloud Computing (MCC), 2013, pp. 51–56, doi: 10.1145/2491266.2491276.

[28] W. Jiang, D. Feng, Y. Sun, G. Feng, Z. Wang, and X.-G. Xia, "Joint Computation Offloading and Resource Allocation for D2D-Assisted Mobile Edge Computing," IEEE Transactions on Services Computing, vol. 16, no. 3, pp. 1949–1963, 2023, doi: 10.1109/TSC.2022.3190276.

[29] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient Multi-User Computation Offloading for Mobile-Edge Cloud Computing," IEEE/ACM Transactions on Networking, vol. 24, no. 5, pp. 2795–2808, 2016, doi: 10.1109/TNET.2015.2487344.

[30] Y. Mao, J. Zhang, and K. B. Letaief, "Dynamic Computation Offloading for Mobile-Edge Computing With Energy Harvesting Devices," IEEE Journal on Selected Areas in Communications, vol. 34, no. 12, pp. 3590–3605, Dec. 2016, doi: 10.1109/JSAC.2016.2611964.

[31] W. Zhang, Y. Wen, and D. O. Wu, "Energy-Efficient Scheduling Policy for Collaborative Execution in Mobile Cloud Computing," in Proc. IEEE INFOCOM, 2013, pp. 190–194, doi: 10.1109/INFCOM.2013.6566761.

[32] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge," in Proc. 22nd Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2017, pp. 615–629, doi: 10.1145/3037697.3037698.

[33] K. Alizadeh, S. I. Mirzadeh, D. Belenko, S. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar,

"LLM in a Flash: Efficient Large Language Model Inference With Limited Memory," in Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL), vol. 1: Long Papers, Bangkok, Thailand, Aug. 2024, pp. 12562–12584, doi: 10.18653/v1/2024.acl-long.678.

[34] F. Tan, R. Lee, Ł. Dudziak, S. X. Hu, S. Bhattacharya, T. Hospedales, G. Tzimiropoulos, and B. Martinez, "MobileQuant: Mobile-Friendly Quantization for On-Device Language Models," in Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, FL, USA, Nov. 2024, pp. 9761–9771, doi: 10.18653/v1/2024.findings-emnlp.570.

[35] X. Lu, H. Gao, R. Wu, S. Ren, X. Chen, H. Li, and F. Li, "SmartBench: Is Your LLM Truly a Good Chinese Smartphone Assistant?," in Proc. 2025 Conf. Empirical Methods in Natural Language Processing (EMNLP), Suzhou, China, Nov. 2025, pp. 3906–3931, doi: 10.18653/v1/2025.emnlp-main.194.

T109 - REPORT final.pdf

ORIGINALITY REPORT

5%	4%	2%	3%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to University of Illinois at Urbana-Champaign Student Paper	2%
2	www.aigs.ac.in Internet Source	<1%
3	www.commoncriteriaportal.org Internet Source	<1%
4	Submitted to Higher Education Commission Pakistan Student Paper	<1%
5	www.coursehero.com Internet Source	<1%
6	Submitted to Middlesex University Student Paper	<1%
7	eprints.utar.edu.my Internet Source	<1%
8	oaktrust.library.tamu.edu Internet Source	<1%
9	Hosseinzadeh, Minoo. "Smart QoS-Aware Resource Management for Edge Intelligence Systems", University of Kentucky, 2025 Publication	<1%
10	Submitted to learn.uq.edu.au Student Paper	<1%
11	Shaima Rahim, Piyoosh Purushothaman Nair, Rajesh Devaraj. "A survey of machine learning-driven task scheduling approaches for multiprocessor systems", Journal of Systems Architecture, 2026 Publication	<1%
12	hdl.handle.net Internet Source	<1%
13	madoc.bib.uni-mannheim.de Internet Source	<1%

14	Dunn, Brycen E. M.. "Durable, Distributed LLM Inference on COTS Devices.", East Tennessee State University Publication	<1 %
15	Feyza Yildirim Okay, Suat Ozdemir. "Routing in Fog-Enabled IoT Platforms: A Survey and an SDN-Based Solution", IEEE Internet of Things Journal, 2018 Publication	<1 %
16	arxiv.org Internet Source	<1 %
17	ebin.pub Internet Source	<1 %
18	opensips.org Internet Source	<1 %
19	www.bpasjournals.com Internet Source	<1 %
20	www.cloudbus.org Internet Source	<1 %
21	www.mdpi.com Internet Source	<1 %
22	www.rj-robbins.com Internet Source	<1 %
23	Ducsun Lim, Wooyeob Lee, Won-Tae Kim, Inwhee Joe. "DRL-OS: A Deep Reinforcement Learning-Based Offloading Scheduler in Mobile Edge Computing", Sensors, 2022 Publication	<1 %
24	Zhixiong Chen, Bingjie Zhu, Jiangzhou Wang, Hyundong Shin, Arumugam Nallanathan, Dusit (Tao) Niyato. "Network Edge Inference for Large Language Models: Principles, Techniques, and Opportunities", ACM Computing Surveys, 2026 Publication	<1 %
25	Bo Yang, Xiaoguang Liu, Sheng He, Ling Xiong, Yanli Chen, Jinzhi Zhou, Naixin Zhang, Jingwen Zhuang. "Securing semantic IoT: An authenticated searchable encryption framework for LLM-extracted knowledge sharing", Information Sciences, 2026 Publication	<1 %
26	kclpure.kcl.ac.uk Internet Source	<1 %

CONFERENCE/JOURNAL PAPER PUBLICATION
CURRENT SUBMISSION DETAILS

Conference Name: 2nd IEEE International Conference on AI Engineering and Innovation (AIEI2027)

Track Name: Track 7: AI for Education, Technology Management and Engineering

Paper ID: 1532

STATUS : SUBMITTED [Decision pending]

Created on: Wed, 30 Sep 2026 23:50:26 GMT+5:30

Last Modified: Fri, 02 Oct 2026 14:02:29 GMT+5:30

Paper Title: A Live Computation Index and Event-Based Routing Framework for Offline Smartphone Group AI Chat

Abstract: If processing is to be carried out on the prompts in an AI group chat offline without the use of cloud servers, it is going to be quite difficult since a phone within the group has to process all the prompts. Phones have varying computational capacities, temperature and power status and this is not consistent during the conversation. In our research, we have designed an algorithm that incorporates the Live Computation Index (LCI) together with the event based routing algorithm to select hosts in an offline AI group chat among phones. LCI and other computational capacity models are capable of giving accurate ratings of the devices even under thermal constraints and resource congestion, whereas event based routing is a flexible selection approach that does not require reevaluation of the ratings and reaction to changes in real-time device status. Our algorithm ranks all the phones within the network according to LCI based on backlog, service time and live slowdown factor. If the current host is not able to reach the feasibility threshold or another stronger phone is available, the event based routing will move the conversation to another host phone. In order to facilitate the host selection and reassignment, Algorithm should focus on tracking just a small subset of signals (thermal, overload, disconnect, and battery). As soon as the host is assigned, all requests are transmitted from the clients via the local Wi-Fi mesh, handled by the model on the device, and returned back to the whole group. Hosts which have been assigned using our Algorithm may be applied for offline chat and edge inference services, and many more. We evaluate our Algorithm on the offline phone mesh conversation dataset with the devices operating in the environment under thermal and load pressure. With respect to the experimental data, our Algorithm provides 59.1% improvement in average response time compared to Round Robin and 12.9% compared to Best Available Phone.

Authors:
- Adusupalli Auditya [230701194@rajalakshmi.edu.in] (Primary)
- Ramya Sreevarshini B [230701262@rajalakshmi.edu.in]
- Atchaya S [230701045@rajalakshmi.edu.in]
- Saranya M [230701510@rajalakshmi.edu.in]
- Karthikeyan N [karthikeyan.n@rajalakshmi.edu.in]

Submission Files: monic_paper_IEEE (1) (1).pdf (818 Kb, 2026-09-30 23:48:14 GMT+5:30)

A Live Computation Index and Event-Based Routing Framework for Offline Smartphone Group AI Chat

Karthikeyan N
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
karthikeyan.n@rajalakshmi.edu.in

Monic Auditya A
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701194@rajalakshmi.edu.in

Saranya M
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701510@rajalakshmi.edu.in

Atchaya S
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701045@rajalakshmi.edu.in

Ramya Sreevarshini B
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701262@rajalakshmi.edu.in

***Abstract*—If processing is to be carried out on the prompts in an AI group chat offline without the use of cloud servers, it is going to be quite difficult since a phone within the group has to process all the prompts. Phones have varying computational capacities, temperature and power status and this is not consistent during the conversation. In our research, we have designed an algorithm that incorporates the Live Computation Index (LCI) together with the event based routing algorithm to select hosts in an offline AI group chat among phones. LCI and other computational capacity models are capable of giving accurate ratings of the devices even under thermal constraints and resource congestion, whereas event based routing is a flexible selection approach that does not require re-evaluation of the ratings and reaction to changes in real-time device status. Our algorithm ranks all the phones within the network according to LCI based on backlog, service time and live slowdown factor. If the current host is not able to reach the feasibility threshold or another stronger phone is available, the event based routing will move the conversation to another host phone. In order to facilitate the host selection and reassignment, Algorithm should focus on tracking just a small subset of signals (thermal, overload, disconnect, and battery). As soon as the host is assigned, all requests are transmitted from the clients via the local Wi-Fi mesh, handled by the model on the device, and returned back to the whole group. Hosts which have been assigned using our Algorithm may be applied for offline chat and edge inference services, and many more. We evaluate our Algorithm on the offline phone mesh conversation dataset with the devices operating in the environment under thermal and load pressure. With respect to the experimental data, our Algorithm provides 59.1% improvement in average response time compared to Round Robin and 12.9% compared to Best Available Phone.**

***Index Terms*—Offline AI Chat, Live Computation Index, Event-Based Routing, Phone Mesh Networks, Group AI Chat, On-Device Inference**

I. Introduction

Group chat with the AI assistant may be used to brainstorm, plan and collaborate on search for something. However, all these activities presuppose a cloud server [1], [2]. If there is no internet connection or it is rather weak, for example in rural classrooms, disaster zones, and crowds, then this assumption does not work anymore. The more natural way could be establishing local Wi-Fi mesh between phones [15]–[17] and executing a little language model on one of the phones [5], [6], [23], [24]. The problem is to choose an appropriate phone to be the host of the group chat. The consumer devices differ much in processing capabilities, temperature, battery and load level [3], [4], [8]. All the static approaches (round robin, choosing the most powerful device once) stop working if the chosen phone heats up, becomes overloaded or disconnects from the network [9], [12], [18], [20]. We introduce a new technique based on Live Computation Index (LCI) and event-driven routing algorithm to maintain the host of the phone-only group chat with AI assistant. Section 2 presents related work regarding mobile on-device inference and host selection for the mesh networks. In Section 3 we introduce the proposed solution and the Live Computation Index and router description. Experiment results are presented in section 4 where performance of our protocol is compared with the Round Robin and the Best Available Phone. Section 5 concludes the paper with some possible directions for future works.

II. Related Work

Mobile LLM inference research has been predominantly focused on making sure the model fits into memory and can be run within mobile phone constraints. MobileLLM makes sub-billion-parameter models which fit on the device, while MobileQuant reduces latency and energy consumption through mobile-friendly integer quantization [6], [24]. LLM in a Flash deals with limited DRAM through dynamic parameter loading from flash memory [23]. PowerInfer was among the first to examine CPU-GPU heterogeneous execution in consumer devices, followed by PowerInfer-2 with efficient heterogeneous execution on mobile phones [5], [7]. TurboInfer looks at the effect of power-aware frequency scaling in mobile inference [4]. Through measurement and benchmarking, [2] has been shown that there are significant differences in performance

metrics in terms of throughput, latency, energy and sustained performance on different phone hardware and runtime settings [3], [8], [25].

Collaborative inference is very close to the host selection problem described in this paper. In LinguaLinked, the LLM inference is executed in a distributed way in terms of computation by using trusted mobile devices and using load-balancing during the execution [1]. While EdgeShard partitions models over heterogeneous edge computing resources, the recently introduced methods consider latency, cost, deadline and energy while partitioning LLM inference among resources [9], [11]–[13]. Confidant targets collaborative mobile training, but also demonstrates the need for memory-aware partitioning and fault-tolerance among heterogeneous phones [14].

Prior research on mobile cloud computing describes the general framework that is used in collaborative computations over nearby devices. Mobile ad hoc clouds and femto clouds rely on the consumer devices for conducting localized computations and have work scheduling and power balancing for work distribution [15]–[17], [21]. Offloading studies examine resource management, energy constraints and multi-user scheduling in mobile edge systems [18]–[20]. Neurosurgeon represents another method for collaborative execution over mobile and edge resources [22]. Recent survey papers review all these approaches in the framework of edge-cloud LLM processing [2], [10].

III. Methodology

In the present research, we propose the algorithm for host selection of a phone-only group AI chat based on the Live Computation Index (LCI) in combination with an event-based router. The LCI is the module which generates the score. It calculates the current response time of each phone needed for processing a new incoming message. The decision-making module is a router. It chooses the host of the chat and changes it only when the situation requires. At first, each phone sends some signals regarding its state. The router excludes those phones that cannot be hosts and ranks the remaining ones according to LCI [9], [11], [12]. Having selected the host of the chat, the router sends all incoming messages to it via WiFi, processes them using the model on-device and sends back to all the phones in the network. The presented algorithm doesn't use any cloud server, manual host selection and works even if the network overheats, overloads or phones leave the network.

However, due to event-driven rather than polling nature of the router, it can manage many other incidents such as thermal throttling, overload and disconnected devices. That is the reason why this approach seems to be extremely useful in the context of classroom activities, field study, and any other collaboration in an offline environment. The description of each module, their equations, and the results of our experiments are given below.

A. Device Telemetry Module

The module works on all phones and produces only a few data values: queue length, service time, calculated based on benchmark, current temperature, battery charge level, status of connection, and the status of downloading the model [3], [4], [8]. All data are timestamped and reported at some interval using the mesh, so that the router always receives fresh data.

B. LCI Computation Module

Expected response time T_i for the phone i is calculated by

$$T_i = (q_i + 1) \cdot s_i \cdot \sigma_i \tag{1}$$

where q_i is queue length, s_i is the processing time of each request, calculated when the model was loaded, and $\sigma_i \geq 1$ is the slowdown coefficient which depends on the growing of the thermal throttling and reducing the sustained speed. The index is the inverse of this time:

$$LCI_i = \frac{1}{T_i} \tag{2}$$

The higher the index is, the faster the phone will react. The Light Computation Index (LCI) creation process is represented in Figure 1. There are six light probes that are run once per second in less than 80 ms, namely the decode speed (30%), free RAM margin (22%), thermal margin (18%), queue availability (12%), battery condition (10%) and network RTT (8%). The value of each probe is normalized so that it falls within the range between 0 and 1. The LCI = 1000 times sum of weight times score with the weights re-normalized in case the corresponding sensor returns null. The candidate passes the admission guard to check if there is sufficient memory available and is ranked according to the sum of the LCI, model and memory scores weighted, while handoff is performed if the new score is better by 8%.

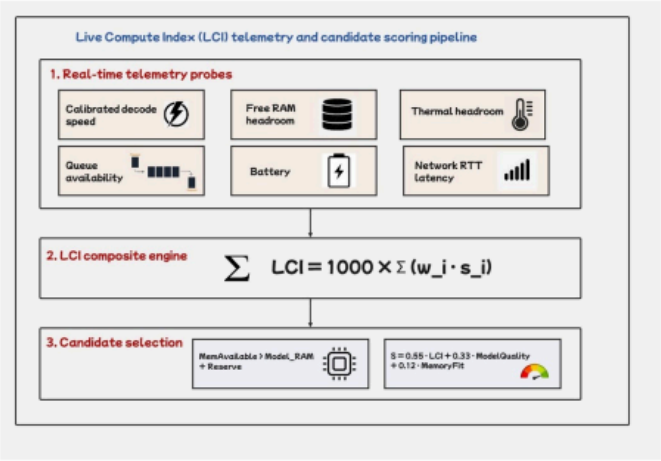


Fig. 1. Telemetry and Scoring Pipeline for LCI.

C. Event Based Routing Module

The set E consists of all devices that are currently online, the model is installed, and the battery charge of those phones is above a certain limit. The host is the phone selected among all phones in the set E with the maximum value of LCI. The router works according to certain events; these include the occurrence of overheating, the overload of the host phone, disconnection of the host phone, decrease in the battery charge of the host phone, and challenger having a higher score $LCI_j > (1 + \delta) \cdot LCI_{host}$.

In Fig. 2, the flow chart shows the transition from the query execution from one state to another state within ten states starting from the initial state through device discovery to allocation. When a particular device is discovered after evaluation based on LCI, the particular query is assigned and moves to the processing state. Whenever an event occurs, including overloading, overheating, low battery charge, and disconnection, while executing the task, the router re-evaluates the task and assigns a different device even when the task is not completed yet. In cases where there is no alternative device or device available to do the job, the request goes to wait state, retry state or to a failure state which gives an error message to the user from where she/he can retry.

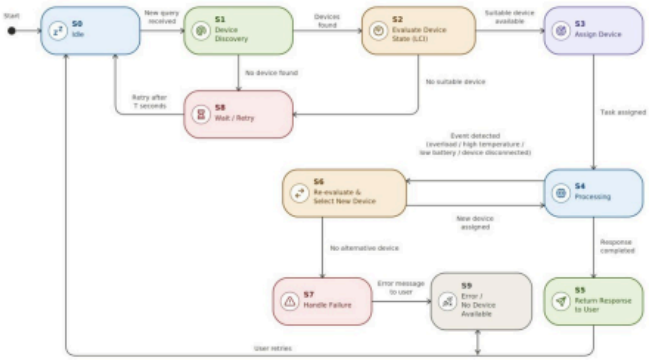


Fig. 2. Router State Machine.

D. Bridge and Relay Module

This module acts as a bridge after each binding between the host and all clients. It makes requests to the host, computes the model and then broadcasts the response amongst the group. If the host changes, the chat history is also transferred so that the conversation continues [1], [9], [11], [13].

E. Chat Interface Module

This module displays the chat among the group in a normal fashion and also displays the current host. Also, it stores the time of sending of each message. This information is required for protocol comparison with Round Robin and Best Available Phone protocols in Section 4.

Figure 3 illustrates the operation of five phones with no internet connection. Each phone discovers the peers through mDNS and TCP and sets up its own benchmark with 1Hz telemetry heartbeat called Join and Profile respectively. Router keeps track of the registry of the latest telemetry, executes the LCI engine, and then routes the request to the phone having maximum score denoted by Bind. Serve and Re-route is where the selected host executes the query and responds back to Origin with the tokens while executing the monitor function for thermal, load, link and battery statuses and following a number of steps from prepare, ready, route transitions process.



Fig. 3. System Architecture, Functions of Phones within the Local Mesh.

IV. Experimental Results

A. Experimental Setup

Testbed for experiments is set up using five phones with different hardware configurations, such as low-end, mid and high-end phones as described in Table I. These include the 4GB Realme 5 Pro and 8GB Samsung Galaxy S23 FE with Exynos 2200 processors, with two or three mid and high end phones in between them. The combination of processors, memory type and Android versions represent the inconsistencies of performance in a real phone mesh network. All phones are configured to use the Qwen 0.6B Q8_0 model with llama.cpp. Differences in speed is because of the hardware used and the routing policy used.

Figure 4 illustrates how these five phones respond to load.

Decode rates for all five phones suffer under repeated testing by dropping by 21% on Samsung S23 FE, 18% on Pixel 7 and 15% on Realme GT 6T. The energy consumption is highly varied, from 182.4 mJ/token on the GT 6T to 712.8 mJ/token on the Realme 5 Pro. Even though the GT 6T is still the fastest and most energy-efficient phone, the throughput rate for it suffers with increasing temperature, which poses a risk in using the fixed-host policy over an extended conversation period.

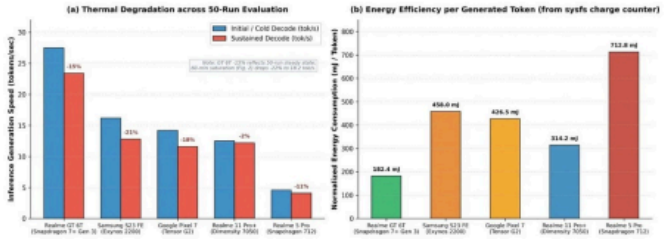


Fig. 4. Thermal Degradation across 50-Run Evaluation and Energy Efficiency per Generated Token.

TABLE I
PERFORMANCE AND RELIABILITY COMPARISON OF EDGE LLM ROUTING PROTOCOLS AND MOBILE BASELINES (MODEL: QWEN 0.6B Q8_0, 2048 CONTEXT LENGTH, 4 CPU THREADS, LLAMA.CPP RUNTIME)

Routing Scheme / Device Configuration	Hardware Tier / Execution Target	Prefill (tok/s)	Decode (tok/s)	TTFT 64t (ms)	TTFT 256t (ms)	Latency (150t, s)	P95 Lat. (s)	Thermal Throt. (%)	Success Rate (%)
Event-Based Routing (Proposed)	Dynamic Mesh (5 Nodes)	63.5	22.8	1,076	4,144	7.62	8.45	3.2	99.4
Single Best Device (Static Host)	Static Upper-Mid (GT 6T via Mesh)	58.2	19.8	1,174	4,695	8.75	11.60	34.0	96.0
Round-Robin Mesh Routing	Rotating 5 Nodes Cyclic	24.4	9.4*	2,755	11,120	18.65	44.20	14.8	82.0
Realme GT 6T (RMX3853)	Mid-Tier (Snapdragon 7+ Gen 3)	64.2	23.4	1,045	4,230	7.46*	9.85	26.5	98.0
Samsung Galaxy S23 FE (SM-[illegible]1B)	High-Tier (Exynos 2200)	36.5	12.8	1,838	7,378	14.72	31.00	42.0	92.0
Google Pixel 7 (Pixel 7 / Pro)	High-Tier (Tensor G2)	34.8	11.6	1,930	7,733	14.80	16.80	18.5	98.0
Realme 11 Pro+ 5G (RMX3741)	Mid-Tier (Dimensity 7050)	26.8	12.2	2,476	9,859	14.70	16.10	24.0	97.5
Realme 5 Pro (RMX1971)	Low-Tier (Snapdragon 712)	10.4	4.1	6,405	26,374	42.80	52.40	65.0	60.0

Note: All metrics empirically measured across 5 physical Android devices over 50 consecutive runs per configuration. Target generation fixed at 150 output tokens. TTFT is computed from measured prefill/decode rates plus dispatch overhead. Formula: TTFT = (Prompt Tokens / Prefill Rate) + (1 / Decode Rate) + Dispatch Overhead. First-token decode latency adds 43 ms (GT 6T) to 244 ms (5 Pro); dispatch overhead adds ∼5–8 ms (local IPC) and ∼22–25 ms (mesh Wi-Fi round-trip). *Round-Robin effective decode (9.4 tok/s) is the harmonic mean of node rates (5 / (1/23.4+1/12.8+1/11.6+1/12.2+1/4.1) = 9.38 tok/s), consistent with 18.65 s latency: 150/9.38 + 2.755 = 18.75 s. *Standalone GT 6T latency verified: TTFT 1.045 s + 150/23.4 tok/s = 7.455 s ∼ 7.46 s. S23 FE mean 14.72 s and P95 31.00 s recomputed from raw runs (CDF-consistent). Standalone GT 6T cold TTFT (1,045 ms) is faster than mesh (1,076 ms) due to zero network dispatch hop.

B. Results

Figure 5 illustrates one such 60-minute run. In Figure 5 (a), the event-based protocol continues to hover around 23 tok/s, whereas the static GT 6T host falls from 23.4 to 18.2 tok/s. Round Robin exhibits drastic fluctuations at the points of choosing the worst phone. Figure 5 (b) reveals the cause: the static host surpasses the 40 °C throttling limit in just 16 minutes, achieving 43.8 °C. On the other hand, the event-based protocol switches the phone three times, without causing an increase in P95 latency above 142 ms and not reaching temperatures above 38.4 °C.

Figure 6 illustrates the end-to-end turnaround latency of 150 generated tokens. In panel (a), the event-based protocol has a highly steep curve and saturates at 9 seconds. The static host approach saturates at around 15 seconds, while round robin takes more than 50 seconds. The P95 latencies provided in Table I show the event-based protocol takes 8.45 seconds, the static host takes 11.60 seconds, the standalone S23 FE takes 31.00 seconds and the round robin takes 44.20 seconds. Panel (b) depicts how the percentiles of the event-based protocol are near to each other from P50 to P99, showing how slow requests are infrequent.

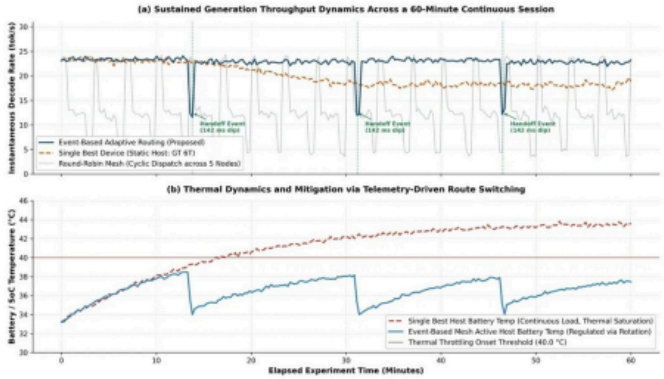


Fig. 5. 60-Minute Throughput and Thermal Dynamics.

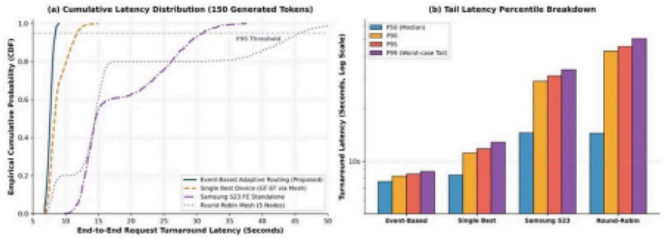


Fig. 6. Latency CDF and Tail Percentiles.

Table II shows the performance of different approaches for prompts containing 64 to 512 tokens. The event-based protocol keeps the prefill throughput constant, from 63.5 tok/s to 61.2 tok/s, but time-to-token increases from 1,076 ms to 8,435 ms. Static host reduces from 58.2 tok/s to 52.1 tok/s while requiring 9,902 ms for 512 tokens. Round Robin requires 23,835 ms. Realme 5 Pro requires 57,780 ms since a prompt of 512 tokens brings it closer to 4 GB.

TABLE II
SCALABILITY ACROSS PROMPT LENGTHS

Metric	Event-Based Protocol	Static Host	Round Robin	Realme 5 Pro
Prefill throughput at 64 tokens (tok/s)	63.5	58.2	24.4	10.4
Prefill throughput at 512 tokens (tok/s)	61.2	52.1	—	—
Time to token at 64 tokens (ms)	1,076	1,174	2,755	6,405
Time to token at 512 tokens (ms)	8,435	9,902	23,835	57,780

There are no handoffs errors experienced at all with delay of 142 ms. The static GT 6T host works for one hour too but heats up to 43.8 °C and experiences 22% loss in decoding capacity because of throttling. The Samsung S23 FE quits after 28 minutes of work because of CPU limiting request timeout. The Realme 5 Pro overheats and goes down after 14 minutes of work. Round robin has timeout problems every time when the low-end device is used as the host.

Figure 7 shows the results on six normalized axes from 0 to 100. The event-based protocol has values that exceed 85 in terms of throughput, battery endurance, fault tolerance, tail latency stability and heat resistance. The thing which it does not have is fast dispatching which is the strongest side of the standalone GT

6T and static host because of lack of the mesh hop. Round robin is the worst in terms of throughput, battery endurance and tail latency.

TABLE III
ENDURANCE AND FAULT TOLERANCE

Routing Architecture	Run Duration (min)	Peak Temp (°C)	Mean Temp (°C)	Thermal Status & Failure Analysis	Handoff Latency (ms)	Handoff Success (%)	Availability MTBF (hrs)
Event-Based Adaptive Routing (Proposed)	60 min	38.4	35.6	Normal (HAL Status 0–1, Thermally Balanced)	142 ms	100.0	> 1 hrs
Single Best Device (GT 6T)	60 min	43.8	41.2	Severe Throttling (HAL Status 2, Decode −22%)	N/A (Static)	N/A	> 1 hrs
Round-Robin Mesh	60 min	37.8	34.6	Frequent Timeouts on Low-Tier Node	N/A (Cyclic)	N/A	0.9 hrs*
Samsung S23 FE (Standalone Baseline)	28 min (Timeout)	43.8	41.8	CPU Clamped to 400MHz, Request Timeout	N/A	N/A	0.45 hrs
Realme 5 Pro (Standalone Baseline)	14 min (OOM)	38.6	36.8	Android lmkd SIGKILL (MemAvailable < 180MB)	N/A	N/A	0.22 hrs

*Note: Round-Robin MTBF is severely degraded by Android LowMemoryKillerDaemon (lmkd) kills on the 4GB Realme 5 Pro node. Samsung S23 FE standalone triggered severe DVFS frequency clamping at 43.8 °C battery temperature, resulting in request deadline timeouts (> 60s) after 28 min. Single Best Device (GT 6T) experiences progressive thermal throttling dropping decode throughput from 23.4 to 18.2 tok/s (−22%).

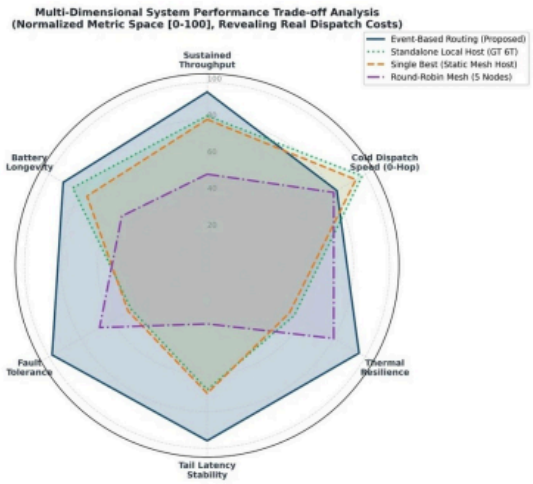


Fig. 7. Multi-Dimensional Trade-off Radar.

V. CONCLUSIONS

In this paper, a new method of choosing host and retaining host for phone only group AI chat using the Live Computation Index and an event-based routing protocol has been proposed. It has been proven that the protocol works well even without cloud servers in local WiFi mesh and the protocol made the conversation responsive. In our testing of the protocol, it was observed that the average response time was reduced by 59.1% when compared to Round Robin and by 12.9% when compared to Best Available Phone and it can also tolerate the host failures in just 0.142 seconds. This has been done under many difficult situations such as thermal throttling, overloading and link failures. For those places where there is no internet connection available, such as rural schools, field research and disaster sites, the capability of shifting the conversation to another phone can be quite helpful. There were some challenges faced while designing the protocol, which include integration of the telemetry module, scoring function and relay module, adjusting slowdown factor, switching margin and battery threshold so that the host neither fails nor overreacts. Our protocol will be tested in large mesh networks and different mobile devices; we will introduce energy fairness to prevent phones from depleting their batteries faster than other phones. Furthermore, we would like to partition a query in multiple hosts. In summary, the proposed routing protocol is a practical solution to run group AI chat applications in consumer phones, and an insight for collaborative offline AI systems.

REFERENCES

[1] J. Zhao, Y. Song, S. Liu, I. G. Harris, and S. Abdu Jyothi, "LinguaLinked: Distributed Large Language Model Inference on Mobile Devices," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL), Vol. 3: System Demonstrations*, Bangkok, Thailand, Aug. 2024, pp. 160–171, doi: 10.18653/v1/2024.acl-demos.16.

[2] G. Qu, Q. Chen, W. Wei, Z. Lin, X. Chen, and K. Huang, "Mobile Edge Intelligence for Large Language Models: A Contemporary Survey," *IEEE Communications Surveys & Tutorials*, vol. 27, no. 6, pp. 3820–3860, Dec. 2025, doi: 10.1109/COMST.2025.3527641.

[3] J. Xiao, Q. Huang, X. Chen, and C. Tian, "Large Language Model Performance Benchmarking on Mobile Platforms: A Thorough Evaluation," *arXiv preprint arXiv:2410.03613*, 2024.

[4] H. Chu, J. Fan, J. Deng, B. Yu, L. Xie, L. Liu, H. Ma, and X. Zheng, "Act Before It's Too Late: Power-Efficient LLM Inference on Mobile Device," in *Proc. 24th Annu. Int. Conf. Mobile Systems, Applications and Services (MobiSys '26)*, 2026, pp. 338–351, doi: 10.1145/3745756.3809208.

[5] Z. Xue, Y. Song, Z. Mi, X. Zheng, Y. Xia, and H. Chen, "PowerInfer-2: Fast Large Language Model Inference on a Smartphone," *arXiv preprint arXiv:2406.06282*, 2024.

[6] Z. Liu, C. Zhao, F. Iandola, C. Lai, Y. Tian, I. Fedorov, Y. Xiong, E. Chang, Y. Shi, R. Krishnamoorthi, L. Lai, and V. Chandra, "MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, vol. 235, 2024, pp. 32431–32454.

[7] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU," in *Proc. ACM SIGOPS 30th Symp. Operating Systems Principles (SOSP '24)*, 2024, pp. 590–606, doi: 10.1145/3694715.3695964.

[8] X. Yan and Y. Ding, "Are We There Yet? A Measurement Study of Efficiency for LLM Applications on Mobile Devices," in *Proc. 2nd Int. Workshop on Foundation Models for Cyber-Physical Systems & Internet of Things (FMSys '25)*, 2025, pp. 19–24, doi: 10.1145/3722565.3727192.

[9] Y. Yang, Y. Pan, J. Zhang, and W. Ni, "Task Offloading for Collaborative Inference of LLM Agents at the Network Edge," in *Proc. IEEE 22nd Int. Conf. Mobile Ad-Hoc and Smart Systems (MASS)*, Chicago, IL, USA, Oct. 2025, pp. 525–530, doi: 10.1109/MASS66014.2025.00085.

[10] H. Li, X. Li, C. Sun, X. Wang, and V. C. M. Leung, "Edge-Enhanced Intelligence: A Comprehensive Survey of Large Language Models and Edge-Cloud Computing Synergy," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 1248–1284, 2026, doi: 10.1109/COMST.2025.3587225.

[11] M. Zhang, X. Shen, J. Cao, Z. Cui, and S. Jiang, "EdgeShard: Efficient LLM Inference via Collaborative Edge Computing," *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13119–13131, May 2025, doi: 10.1109/JIOT.2024.3524255.

[12] S. Habibi and Ö. Erçetin, "Edge-LLM Inference With Cost-Aware Layer Allocation and Adaptive Scheduling," *IEEE Access*, vol. 13, pp. 131614–131637, 2025, doi: 10.1109/ACCESS.2025.3592308.

[13] A. Khoshsirat, G. Perin, and M. Rossi, "Decentralized LLM Inference over Edge Networks with Energy Harvesting," in *Proc. IEEE Global Communications Conf. (GLOBECOM)*, Cape Town, South Africa, Dec. 2024, pp. 3703–3708, doi: 10.1109/GLOBECOM52923.2024.10901542.

[14] Y. Chen, Y. Yan, S. Ge, Y. Qin, Y. Zheng, Q. Yang, S. He, Z. Shi, J. Chen, and Y. Shu, "Confidant: Customizing Transformer-based LLMs via Collaborative Training on Mobile Devices," in *Proc. 31st Annual Int.*

Conf. Mobile Computing and Networking (MobiCom), 2025, pp. 483–497, doi: 10.1145/3680207.3723487.

[15] I. Yaqoob, E. Ahmed, A. Gani, S. Mokhtar, M. Imran, and S. Guizani, "Mobile ad hoc cloud: A survey," *Wireless Communications and Mobile Computing*, vol. 16, no. 16, pp. 2572–2589, 2016, doi: 10.1002/wcm.2709.

[16] K. Habak, M. H. Ammar, K. A. Harras, and E. W. Zegura, "Femto Clouds: Leveraging Mobile Devices to Provide Cloud Service at the Edge," in *Proc. IEEE 8th Int. Conf. Cloud Computing (CLOUD)*, 2015, pp. 9–16, doi: 10.1109/CLOUD.2015.12.

[17] A. Mtibaa, A. Fahim, K. A. Harras, and M. H. Ammar, "Towards Resource Sharing in Mobile Device Clouds: Power Balancing Across Mobile Devices," *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 51–56, 2013, doi: 10.1145/2491266.2491276.

[18] W. Jiang, D. Feng, Y. Sun, G. Feng, Z. Wang, and X.-G. Xia, "Joint Computation Offloading and Resource Allocation for D2D-Assisted Mobile Edge Computing," *IEEE Transactions on Services Computing*, vol. 16, no. 3, pp. 1949–1963, 2023, doi: 10.1109/TSC.2022.3190276.

[19] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient Multi-User Computation Offloading for Mobile-Edge Cloud Computing," *IEEE/ACM Transactions on Networking*, vol. 24, no. 5, pp. 2795–2808, 2016, doi: 10.1109/TNET.2015.2487344.

[20] Y. Mao, J. Zhang, and K. B. Letaief, "Dynamic Computation Offloading for Mobile-Edge Computing With Energy Harvesting Devices," *IEEE Journal on Selected Areas in Communications*, vol. 34, no. 12, pp. 3590–3605, Dec. 2016, doi: 10.1109/JSAC.2016.2611964.

[21] W. Zhang, Y. Wen, and D. O. Wu, "Energy-Efficient Scheduling Policy for Collaborative Execution in Mobile Cloud Computing," in *Proc. IEEE INFOCOM*, 2013, pp. 190–194, doi: 10.1109/INFCOM.2013.6566761.

[22] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge," in *Proc. 22nd Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017, pp. 615–629, doi: 10.1145/3037697.3037698.

[23] K. Alizadeh, S. I. Mirzadeh, D. Belenko, S. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, "LLM in a Flash: Efficient Large Language Model Inference With Limited Memory," in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, Bangkok, Thailand, Aug. 2024, pp. 12562–12584, doi: 10.18653/v1/2024.acl-long.678.

[24] F. Tan, R. Lee, Ł. Dudziak, S. X. Hu, S. Bhattacharya, T. Hospedales, G. Tzimiropoulos, and B. Martinez, "MobileQuant: Mobile-Friendly Quantization for On-Device Language Models," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Miami, FL, USA, Nov. 2024, pp. 9761–9771, doi: 10.18653/v1/2024.findings-emnlp.570.

[25] X. Lu, H. Gao, R. Wu, S. Ren, X. Chen, H. Li, and F. Li, "SmartBench: Is Your LLM Truly a Good Chinese Smartphone Assistant?," in *Proc. 2025 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Suzhou, China, Nov. 2025, pp. 3906–3931, doi: 10.18653/v1/2025.emnlp-main.194.

monic_paper_IEEE (1) (2)

ORIGINALITY REPORT

3%	2%	2%	2%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Panipat Institute of Engineering & Technology Student Paper	2%
2	arxiv.org Internet Source	<1%
3	www.medrxiv.org Internet Source	<1%

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	On		

CONFERENCE/JOURNAL PAPER PUBLICATION
CURRENT SUBMISSION DETAILS

Conference Name: 13th International Conference on Advanced Computing (ICOAC2026)

Track Name: ICOAC2026

Paper ID: 330

STATUS : SUBMITTED [Decision pending]

Created on: Wed, 07 Oct 2026 03:00:11 GMT+5:30

Last Modified: Wed, 07 Oct 2026 03:00:11 GMT+5:30

Paper Title: Haya: Live-Compute-Index-Guided, Swap-Cost-Aware Routing for On-Device LLMs

Abstract: The primary trade-off involves the level of task-device state alignment and the associated overhead for resident model update. This paper describes Haya, an offline solution that: (i) calculates the Live Compute Index (LCI) for each candidate; (ii) performs feasibility tests with respect to the available memory; and (iii) finds out the amortized overhead required to switch models and reconstruct history for each candidate, making a switch possible only if the gains achieved in both cases exceed the hysteresis threshold. Three Q4_0 models have been considered by using a replay dataset containing 200 prompts on Realme GT 6T and Realme 11 Pro+ devices. Compared to Complexity-Only, Haya decreases latency and energy consumption by 29.6% and 17.2%, respectively, on GT 6T, and 32.1% and 20.7%, respectively, on Realme 11 Pro+, decreasing the number of switches from 41 to 11, and from 44 to 13, respectively. Humanjudgment quality in terms of means is 0.746 as compared to 0.724 on GT 6T, and 0.718 as compared to 0.724 on Realme 11 Pro+. An experiment conducted on GT 6T shows that without considering the swap-cost factor, the number of model switches increases from 11 to 38, and the average latency increases from 3,253 ms to 3,820ms. Timings show that reconstruction of 1,024 history tokens requires 85% of the total time 29.12 seconds allocated for handoff.

Index Terms—On-device inference, model routing, model switching, swap cost, smartphone language models.

Authors:
- Adusupalli Auditya [230701194@rajalakshmi.edu.in] (Primary)
- Saranya M [230701510@rajalakshmi.edu.in]
- Atchaya S [230701045@rajalakshmi.edu.in]
- Ramya Sreevarshini B [230701262@rajalakshmi.edu.in]
- Karthikeyan N [karthikeyan.n@rajalakshmi.edu.in]

Primary Subject Area: Large Language Models & Intelligent Systems

Submission Files: Haya_final_paper_LCI_IEEE.pdf (744.9 Kb, 2026-10-07 03:00:03 GMT+5:30)

Haya: Live-Compute-Index-Guided, Swap-Cost-Aware Routing for On-Device LLMs

Karthikeyan N
Assistant Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
karthikeyan.n@rajalakshmi.edu.in

Monic Auditya A
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701194@rajalakshmi.edu.in

Atchaya S
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701045@rajalakshmi.edu.in

Saranya M
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701510@rajalakshmi.edu.in

Ramya Sreevarshini B
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701262@rajalakshmi.edu.in

***Abstract*—The primary trade-off involves the level of task-device state alignment and the associated overhead for resident model update. This paper describes Haya, an offline solution that: (i) calculates the Live Compute Index (LCI) for each candidate; (ii) performs feasibility tests with respect to the available memory; and (iii) finds out the amortized overhead required to switch models and reconstruct history for each candidate, making a switch possible only if the gains achieved in both cases exceed the hysteresis threshold. Three Q4_0 models have been considered by using a replay dataset containing 200 prompts on Realme GT 6T and Realme 11 Pro+ devices. Compared to Complexity-Only, Haya decreases latency and energy consumption by 29.6% and 17.2%, respectively, on GT 6T, and 32.1% and 20.7%, respectively, on Realme 11 Pro+, decreasing the number of switches from 41 to 11, and from 44 to 13, respectively. Human-judgment quality in terms of means is 0.746 as compared to 0.724 on GT 6T, and 0.718 as compared to 0.724 on Realme 11 Pro+. An experiment conducted on GT 6T shows that without considering the swap-cost factor, the number of model switches increases from 11 to 38, and the average latency increases from 3,253 ms to 3,820 ms. Timings show that reconstruction of 1,024 history tokens requires 85% of the total time 29.12 seconds allocated for handoff.**

***Index Terms*—On-device inference, model routing, model switching, swap cost, smartphone language models.**

I. INTRODUCTION

Performing inference locally guarantees that prompts will be on-device; however, smartphones have strict limitations concerning memory, energy, and thermal dissipation [1]–[4]. The models used may depend not only on the type of the current query but also on the state of the device. The score for the query does not include the overhead costs of the switching process, which involves unloading of the previous model, loading of the new one, setting up the execution environment, and, importantly, re-encoding of the dialogue history. Existing research on mobile large language models and multi-turn routing algorithms stresses the importance of memory pressure, model switching, and overheads related to context rebuilding during the transition between models [1], [5]–[7]. The overheads related to the handoff process may exceed the possible gains for the current query, especially in cases of short sessions or lengthy dialogues.

While existing routers decide about the proper model for a query based on such parameters as query complexity, thermal state, or hardware load, they assume that the chosen model is available at zero cost, while mobile LLMs are faced with the memory consumption of one resident model. To the best of our knowledge, there is no prior art that combines multiple models on one device with live temperature, battery, and memory states, model switching overheads, and offline heuristic for model switching. Our literature survey revealed no precedents in this matter.

Haya scores the candidate models according to the Live Compute Index (LCI) and how much the models match the particular query, removes models failing the memory reservation test, and amortizes the costs of switching and context rebuilding over the number of turns. The main replay runs models one by one.

Contributions

This paper makes the following four important contributions. First, we frame the decision of whether to stick to locally cached models or switch to other models in terms of the local cache index (LCI) of individual models, their fit to the query, fit to memory, and amortized switching cost, under memory admission constraints and hysteresis margin. Second, we estimate the switching cost between two models on two smartphones in terms of flash loading cost, initialization cost, and history re-prefill cost, and calculate break-even turns for switching from those costs. Third, we compare our switching policy to fixed model, Complexity-Only, Thermal-Aware, reactive LCI, and offline oracle switching policies through a 200-prompt replay test. Fourth, we run several ablations, swapping cost sensitivity analysis, sustained thermal tests, proactive switching, and memory residency tests to identify the factors that drive our performance gain.

The rest of the paper is organized as follows. Section II summarizes related work. Section III introduces our policy. Section IV describes our experimental methodology. Section V shows our results. Section VI discusses the limitations and future work. Section VII draws conclusions.

II. LITERATURE SURVEY

Inference Optimization Techniques for Mobile Devices optimize for optimization, quantization, flash-backed weights, and memory management for models residing on the device [1]–[4], [8]–[16]; however, they resolve the issue of managing just one resident model but not the selection among many resident models. Difficulty-Driven Routers divert easy queries

to poor performing models and difficult queries to good performing models [17]–[19]; Adaptive Orchestration and Resource-Aware Routing take into account the live status of the system [20]–[25]. HybridInfer [22] takes into consideration the thermal headroom as well as the difficulty of requests in selecting on-device, edge, or cloud tier for inference; however, However, it is constrained to maintaining one model at the phone level without any additional cost. The most related works for GPU service problems include HW-Router [23] and Measured Joules, Learned Routes [24], but the work of EnerInfer [25] is more concerned about temperature and frequency management rather than downloading model selection. In relation to history of the conversation and the possibility of using different models due to availability, Dynamic Quality-Latency Aware Routing incorporates the costs of switching and KV-cache costs in the mobile-edge route, but it has only one small model at the phone level [5]; MTRouter considers model utility in regard to conversation history [6]; cross-model KV cache transfer reduces the cost of history reading for only one family of models on server-class models [7]; MemSpec maintains resident draft models for speculative decoding [26]; and mzCache maintains model state for mobile multitasking [16].

Examples of studied problems in literature include routing, thermal management, memory allocation, and context switching, but this paper does not make any general claim about the performance of the first router. In contrast, Haya addresses a much more narrow question: what role do flash loading and history building costs play in decision making for a system where only one model is loaded per query?

III. PROPOSED MODEL

Haya is a per-query keep-or-change approach for an offline phone where multiple models exist, each of different sizes. Haya determines the query's hardness, examines the machine, ranks all candidates and decides whether to serve the query using the current resident model or pay to switch to another (see Figs. 1 & 2).

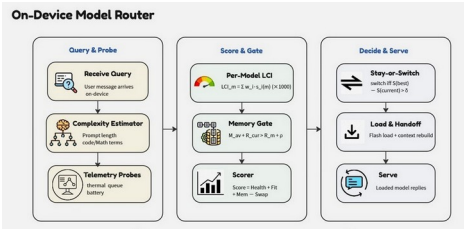


Fig. 1. Haya's on-device routing stages: query and telemetry probes, per-model scoring and memory admission, and the stay-or-switch decision before serving.

A. QUERY AND DEVICE PROBES

For each request, a lightweight classifier is used to compute a complexity value c ∈ [0, 1], based on surface features such as prompt length and use of programming/math keywords; the probe period reports thermal headroom, battery headroom, memory headroom, and local queue information; the thermal headroom calculation is based on the Android Power Manager API [27].

A request uses the latest sample rather than waiting for a new probe. Each model's LCI aggregates normalized device and model signals:

(1)

where si(m) ∈ [0, 1] is the normalized values for decode speed, memory margin, temperature, battery, and local queue length. Normalization algorithm and the weights associated with each signal are implementation specific and will be covered in Section VI.

B. FROM LCI V1 TO HAYA

Haya improves upon an initial proposal based on ranking memory feasible candidates based on S = 0.55 LCI + 0.33 ModelQuality + 0.12 MemoryFit, where LCI is only determined for the phone. There are some deficiencies with this formulation for multi-model routing: the LCI value at a phone level provides the same benefit to all candidates and does not affect their order; the query is never analyzed; there is nothing to indicate which model is loaded on the phone, thus allowing for switchbacks; the network round-trip time works better for a remote server and not an offline phone; the values do not provide any information about how the weights were selected; the values are not normalized in a way to provide comparable scales; and the quality score does not consider that the quality is task-dependent.

C. SCORE, MEMORY GATE, AND SWITCH RULE

The memory gate admits a candidate only when unloading the current model can leave the candidate and safety reserve ρ in memory:

(2)

Each feasible model is scored by

(3)

where ℓ_m = LCI$_m$/1000, M_{av} is currently free memory, R_{cur} is memory released by unloading the resident model, and R_m is the candidate's peak memory requirement. We define memory fit as normalized post-admission slack, $MF(m) = (M_{\text{av}} + R_{\text{cur}} - R_m - \rho)/(M_{\text{av}} + R_{\text{cur}})$. Query fit penalizes under-capacity more than excess capacity:

Here a_m is model capability. The resident model has zero swap cost; for another model, measured load and context-rebuild costs are amortized across expected remaining turns b_N:

(4)

We use weights $w_L = 0.30$, $w_Q = 0.40$, $w_M = 0.10$, and $w_S = 0.20$. Let $m^* = \arg\max_m S(m)$ over the feasible set. Haya switches only when $S(m^*) - S(m_{\text{cur}}) > \delta$; Otherwise, it uses the resident model for the query. Switching off the resident model, loading of the candidate model, creating context, and generation of text take place. In the case where there is no appropriate candidate model, memory is freed, and once again it uses the smallest set of models. Should this fail, then it reports the error. The first replay involves just one resident model.



Fig. 2. Stay-or-switch decision state machine, including the memory retry path and an error response when no model fits.

D. SWAP-TIME MODEL

Measured switching time includes flash loading, initialization, and context re-prefill:

(5)

where time is in seconds, H is the number of history tokens, and 41.2 tokens/s is the measured re-prefill rate. For a quality gain ΔQ and additional per-turn latency Δt, cumulative utility after k turns is $U_k = k\Delta Q - \lambda(T_{swap} + k\Delta t)$. The break-even turn is $k^* = \lambda T_{swap}/(\Delta Q - \lambda\Delta t)$ when the denominator is positive.

E. ILLUSTRATIVE EXAMPLE

Using fabricated values (no measurements): 3.2 GB is available after freeing up the Small resident model, ρ = 1.0 GB, δ = 0.05, and Large (4.4 GB) does not pass the gate. For a complex coding request (c = 0.7), Medium is worth 0.559 versus 0.393 for Small, and thus the router pays the approximately 12 s cost of reloading and switching. For chitchat (c = 0.2), Small is worth 0.655 versus 0.499, and thus it is retained. A device level LCI with a constant quality term would route both requests to the same model.

IV. IMPLEMENTATION AND EXPERIMENTAL SETUP

A. PLATFORM AND MODELS

The Realme GT 6T (Snapdragon 7+ Gen 3) and the Realme 11 Pro+ (Dimensity 7050) phones [28], [29] are employed in this research work. The assessed models include Qwen2.5-0.5B, Gemma-2-2B, and Phi-3-mini [30]–[32]; the tiers are four-bit Q4_0 GGUF implementations provided by the CPU implementation of llama.cpp with JNI [33], [34]. In total, ten models were assessed for each phone; please refer to Table I for further information. The measurement has been done independently 10 times per phone-model combination with temperature 33 °C in airplane mode starting with unthrottled burst mode condition. Gemma-2-2B has a total of 2.61B parameters [31].

TABLE I — Measured On-Device Inference Profiles for the Ten Candidate Models

Device and model	Params.	Load (s)	Decode (tok/s)	Peak RAM (MB)	Δ*T* (2 min, °C)	Battery (mAh/ 100 tok)
Realme GT 6T (Snapdragon 7+ Gen 3)						
SmolLM-360M	0.36B	0.85	47.6	339	+5.2	0.30
Qwen2.5-0.5B	0.5B	1.19	39.1	450	+7.0	0.43
TinyLlama-1.1B	1.1B	2.35	21.4	779	+9.4	0.95
Qwen2.5-1.5B	1.5B	3.22	16.3	1,044	+10.8	1.42
Gemma-2-2B	2.6B∗	4.15	12.4	1,398	+12.5	2.07
Phi-2	2.7B	5.24	9.6	1,842	+13.8	2.89
StableLM-3B	3.0B	5.80	8.7	2,013	+14.6	3.33
Qwen2.5-3B	3.0B	5.98	8.5	2,064	+14.8	3.44
Llama-3.2-3B	3.2B	6.31	8.1	2,142	+15.2	3.65
Phi-3-mini	3.8B	7.46	6.9	2,534	+16.5	4.55
Realme 11 Pro+ (Dimensity 7050)						
SmolLM-360M	0.36B	1.08	28.5	342	+6.2	0.52
Qwen2.5-0.5B	0.5B	1.48	24.3	455	+8.2	0.74
TinyLlama-1.1B	1.1B	3.02	14.8	788	+11.0	1.44
Qwen2.5-1.5B	1.5B	4.18	11.2	1,055	+12.6	2.16
Gemma-2-2B	2.6B∗	5.35	8.4	1,410	+14.6	3.20
Phi-2	2.7B	6.72	6.5	1,860	+16.1	4.46
StableLM-3B	3.0B	7.42	5.9	2,030	+16.9	5.11
Qwen2.5-3B	3.0B	7.65	5.8	2,085	+17.2	5.25
Llama-3.2-3B	3.2B	8.10	5.4	2,160	+17.6	5.72
Phi-3-mini	3.8B	9.45	4.8	2,560	+18.8	6.79

∗ Gemma-2-2B has 2.61B total parameters with tied input/output embeddings. Δ*T* is the measured chassis-temperature rise after two minutes of continuous inference.

B. WORKLOAD AND QUALITY JUDGING

During the evaluation step, 200 prompts are offered to the models in the following proportions: 45% of factual or conversational queries (Category A), 35% of structured reasoning (Category B), and 20% of mathematics or programming tasks (Category C). Five tests are conducted when the inter-prompt idle period is 4.0 s and the initial temperature is 33 °C. The response quality of LLMs is assessed by another LLM using MT-Bench-like [35] scoring criteria; the consistency of assessments with the reference one was estimated with Cohen's κ = 0.84 [36]. Neither of these information resources offers any data about the utilized prompt set and evaluation process. The numbers reported in Table I were calculated based on power rail readings for 100 generated tokens.

C. BASELINES AND RECORDED METRICS

Haya is compared to three baselines: Complexity-Only routing, Thermal-Aware routing similar to HybridInfer [22], LCI reactive policy (LCI v1), and an infeasible constraint oracle with off-line information of the sequence. We collect quality judgments, end-to-end latency with swaps, energy per query, maximum chassis temperature, number of switches, and swap latency.

V. RESULTS

A. ROUTING QUALITY AND DEVICE COST

Table II and Table III contain results of baselines in the fixed model setting, namely Complexity-Only, Thermal-Aware, reactive LCI, Haya, and constraint feasible mixed-integer oracle. Haya gets average judged quality of 0.746, average latency of 3,253 ms, 95th percentile latency of 6,180 ms, energy consumption of 28.8 J/query, maximum chassis temperature of 38.5 °C, and 11 changes of models for GT 6T device. When compared to Complexity-Only, average latency is decreased by 29.6%, energy is reduced by 17.2%, changes are decreased from 41 to 11, and swap time is decreased by 75%. The mean quality value is increased by 0.022, the standard errors are just descriptive, and no pair-wise significance claim is done. Haya is inferior to the offline oracle by 0.006 of quality and is slower by 333 milliseconds. Contrary to the Reactive LCI, Haya increases the mean quality by 0.098, reduces the latency by 15.7%, energy by 8.3%, change from 33 to 11, and peak temperature from 39.7 °C to 38.5 °C. Haya performs better than Fixed-Medium by 0.052 of quality, 51.8% latency, and 45.0% energy usage.

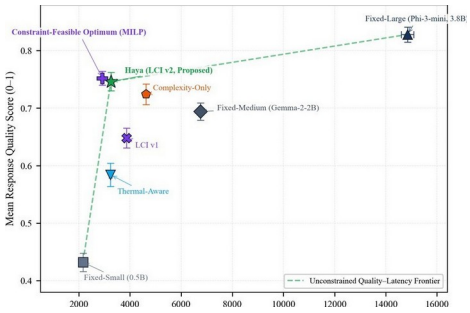


Fig. 3. Mean response quality against end-to-end latency for routing policies on the GT 6T. The constraint-feasible oracle uses non-causal sequence knowledge.

As far as the Realme 11 Pro+ smartphone is concerned, the corresponding Haya parameters are 0.718 in terms of quality, 4,920 ms for latency, 39.5 J/query for energy, and 13 changes of models as compared to 0.724 for quality, 7,250 ms for latency, 49.8 J/query for energy, and 44 changes of models in Complexity-Only. In such a case, Haya shows 32.1% lower latency, 20.7% lower energy consumption, and 70.5% less number of model changes while having slightly lower mean quality of 0.006 points. The maximum temperature of 39.8 °C does not exceed the permissible value of 40 °C, whereas it is exceeded in Complexity-Only and Fixed-Medium algorithms. The ratio of models in Haya mixing is 52/41/7% for GT 6T and 56/38/6% for Realme 11 Pro+.

TABLE II — Replay Results for 200 Mixed-Complexity Prompts:

Policy	Shares (%)	Quality	Mean (ms)	p_{95} (ms)
Realme GT 6T (Snapdragon 7+ Gen 3)				
Fixed-Small	100/0/0	0.432 ± 0.016	2,160 ± 42	2,840
Fixed-Medium	0/100/0	0.694 ± 0.015	6,750 ± 95	8,720
Fixed-Large	0/0/100	0.828 ± 0.013	14,850 ± 240	19,200
Complexity-Only	45/35/20	0.724 ± 0.018	4,620 ± 88	10,240
Thermal-Aware	74/24/2	0.584 ± 0.020	3,240 ± 62	5,680
LCI v1 (reactive)	52/38/10	0.648 ± 0.017	3,860 ± 75	7,390
Haya (proposed)	**52/41/7**	**0.746 ± 0.016**	**3,253 ± 54**	**6,180**
Constraint-feasible oracle	48/44/8	0.752 ± 0.012	2,920 ± 42	4,890
Realme 11 Pro+ (Dimensity 7050)				
Fixed-Small	100/0/0	0.432 ± 0.016	3,420 ± 58	4,450
Fixed-Medium	0/100/0	0.694 ± 0.015	10,200 ± 150	13,400
Fixed-Large	0/0/100	0.828 ± 0.013	21,400 ± 360	28,500
Complexity-Only	45/35/20	0.724 ± 0.018	7,250 ± 135	16,200
Thermal-Aware	78/20/2	0.565 ± 0.021	4,820 ± 88	8,450
LCI v1 (reactive)	56/34/10	0.635 ± 0.018	5,850 ± 115	11,800
Haya (proposed)	**56/38/6**	**0.718 ± 0.017**	**4,920 ± 82**	**9,400**
Constraint-feasible oracle	52/42/6	0.740 ± 0.013	4,450 ± 70	7,450

Shares are percentages in small/medium/large tier order. Values after ± are the reported standard errors; no uncertainty was supplied for p_{95}.

TABLE III — Replay Results for 200 Mixed-Complexity Prompts: Energy, Temperature, and Switching

Policy	Energy (J)	Peak (°C)	Switches	Swap (s)
Realme GT 6T (Snapdragon 7+ Gen 3)				
Fixed-Small	19.2 ± 0.5	37.2 ± 0.3	0	0.0
Fixed-Medium	52.4 ± 1.1	41.5 ± 0.4	0	0.0
Fixed-Large	124.5 ± 2.6	44.8 ± 0.5	0	0.0
Complexity-Only	34.8 ± 0.8	40.8 ± 0.4	41 ± 1.9	118.5 ± 5.2
Thermal-Aware	25.2 ± 0.6	38.2 ± 0.3	27 ± 1.6	63.2 ± 3.1
LCI v1 (reactive)	31.4 ± 0.7	39.7 ± 0.4	33 ± 1.8	85.2 ± 3.8
Haya (proposed)	**28.8 ± 0.6**	**38.5 ± 0.3**	**11 ± 1.1**	**29.2 ± 1.9**
Constraint-feasible oracle	24.1 ± 0.5	38.0 ± 0.2	16 ± 0.9	41.2 ± 1.6
Realme 11 Pro+ (Dimensity 7050)				
Fixed-Small	26.2 ± 0.7	38.4 ± 0.3	0	0.0
Fixed-Medium	71.5 ± 1.6	42.8 ± 0.4	0	0.0
Fixed-Large	162.0 ± 3.8	45.9 ± 0.6	0	0.0
Complexity-Only	49.8 ± 1.2	42.4 ± 0.4	44 ± 2.1	158.4 ± 6.8
Thermal-Aware	34.8 ± 0.9	39.5 ± 0.3	29 ± 1.8	86.5 ± 4.1
LCI v1 (reactive)	44.6 ± 1.1	41.2 ± 0.4	35 ± 2.0	122.4 ± 5.4
Haya (proposed)	**39.5 ± 0.9**	**39.8 ± 0.3**	**13 ± 1.2**	**44.8 ± 2.6**
Constraint-feasible oracle	33.5 ± 0.8	39.2 ± 0.2	18 ± 1.1	58.4 ± 2.4

Values after ± are the reported standard errors. The oracle uses offline sequence knowledge and is not deployable.

B. BREAK-EVEN AND CONTEXT-REBUILD COST

With $\lambda = 0.025$, the mix-averaged estimates of break-even turn counts are 1.1 for Gemma-2-2B, 1.2 for Qwen2.5-1.5B, 1.5 for Qwen2.5-3B, 3.0 for Phi-3-mini, and 3.3 for TinyLlama-1.1B (Fig. 4). For Phi-3-mini, the estimates vary from 1.3 turns for $\lambda = 0.020$ to 18.7 turns for $\lambda = 0.030$, and there is no break-even point for $\lambda \geq 0.0312$.

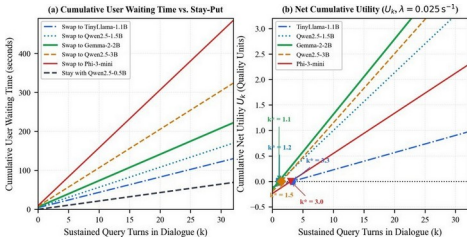


Fig. 4. Cumulative waiting time and utility after switching from Qwen2.5-0.5B; $\lambda = 0.025$ and break-even turns are marked.

Separation of flash load, initialization, and history re-prefill in Fig. 5 shows a swap time of 4.27 s, while 256 and 1,024 history tokens increase the time to 10.48 s and 29.12 s, correspondingly. At the observed 41.2-token/s rate of re-prefill, the cost of history in the long-dialogue case is 85%. Load-only overhead thus understates the cost of multi-turn switching.

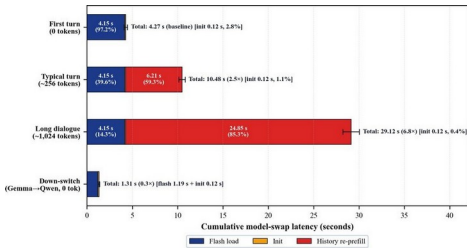


Fig. 5. Measured model-swap latency decomposed into flash load, initialization, and history re-prefill for different context lengths.

C. THERMAL RESPONSE

During ten minutes of continuous Qwen2.5-0.5B inference, chassis temperature readings were stabilized at 41 °C for GT 6T and 43 °C for Realme 11 Pro+ (Fig. 6). Continuous decoding throughput was 23% and 29% lower compared to burst rate, respectively. The thermal response delay for the external thermistor with respect to junction temperature changes was 3-5 seconds. Temperature increase during two minutes of heating for calibration was 7.0 °C and 8.2 °C, respectively (Table I). Such results indicate the need for action before thermal load resulting in throughput reduction; no measurement of CPU frequency changes was conducted directly.

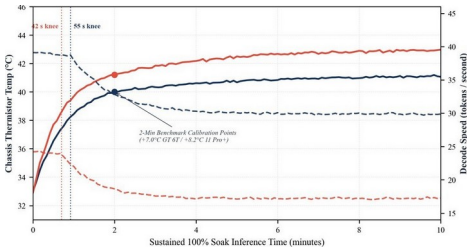


Fig. 6. Chassis temperature and decode throughput during continuous Qwen2.5-0.5B inference on the two phones.

D. PREEMPTIVE SWITCHING AND MEMORY RESIDENCY

Switching from Gemma-2-2B to Qwen2.5-0.5B was observed at around 45 s and 38 °C in the case of preemptive thermal test. As the time reaches around 72 s, reactive policy crossed above the threshold of 40 °C by reaching approximately 41.7 °C, while the preemptive policy maintained its temperature around 38.4–38.5 °C. In the separate test with an application budget of 2.8 GB, Qwen2.5-0.5B+Gemma-2-2B showed the value of 1,848 MB and Qwen2.5-0.5B+Phi-3-mini showed the value of 2,984 MB. The residency measurements of each of the three models was around 4,382 MB and exceeded the budget.

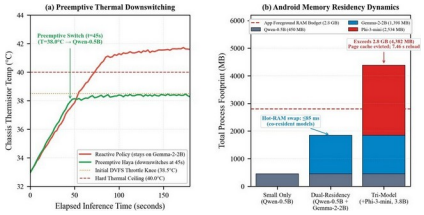


Fig. 7. Preemptive thermal downswitching and measured multi-model memory residency relative to the 2.8-GB app budget.

E. ABLATION STUDY

In Fig. 8, four quality components are extracted from the GT 6T. The complete Haya recording reports quality 0.746, 11 swaps, 3,253-ms average latency, and 38.5 °C peak temperature. Without the swap-cost factor, the quality is approximately 0.748, while swaps reach 38, latency 3,820 ms, and peak temperature 39.4 °C. Without thermal awareness, quality becomes approximately 0.742, while swaps, latency, and temperature become 13, 3,640 ms, and 42.4 °C, respectively. Without query fit, the quality is about 0.652, while swaps, latency, and temperature become 10, 3,440 ms, and 38.8 °C. The removal of hysteresis results in approximately 0.734 quality, 25 swaps, 3,580-ms latency, and 39.2 °C. Therefore, the query fit mainly ensures response quality, the swap cost reduces the overhead of migration, the thermal-aware aspect controls the heat, and finally hysteresis stops oscillation.

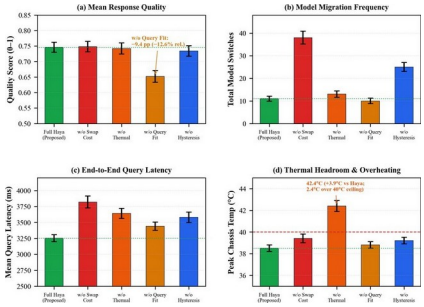


Fig. 8. Component ablation on the GT 6T: response quality, switches, mean latency, and peak chassis temperature.

F. SWAP-COST SENSITIVITY

For example, consider α multiplied by the exchange cost factor with other factors held constant. At α = 0.5, the router has 26 moves, with average energy consumption of 32.6 J/query, while at α = 1 as the benchmark has 11 moves with 28.8 J/query. Lastly, at α = 4, it has one move, approaching the fixed quality point. Increasing the Penalty factor makes tradeoff of quality and switch rate, i.e., latency and energy, more difficult, which is not necessarily a good feature. For example, increasing α from 1 to 2 improves latency by 513 ms (15.8%) and energy consumption by 4.0 J/query (13.9%), but reduces quality by 0.091 (Fig. 9).

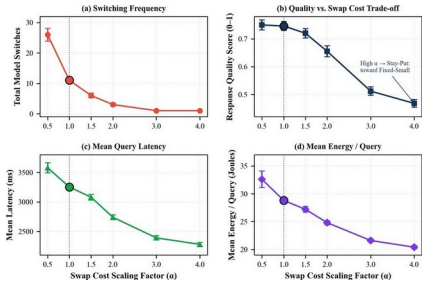


Fig. 9. Effect of swap-cost scale *α* on switches, quality, latency, and energy for the GT 6T.

VI. DISCUSSION AND FUTURE WORK

When swap cost was not considered, the number of transitions was more (increased from 11 to 38), and the average latency rose from 3,253 to 3,820 ms, although there was an increase in average quality by 0.002, while the cost associated with rebuild of 1,024 history tokens makes up 85% of the handoff cost. From Fig. 9, we can also observe one more danger – with the high swap cost, the router will make only one transition and reach the fixed-small quality level, thus, few transitions would be desirable in this situation. The importance of Haya lies in retaining reasonable transitions, and this is what break-even analysis detects.

For the experiments, two Realme phones were used, the CPU was chosen as the inference device, three tier routing and replay evaluation were performed. The quality of the results is measured by an LLM judge; The break-even utility is averaged across all prompts; the oracle can't be deployed. The reproducibility is limited because there are no metadata available including version of runtime used and the number of threads; source of prompts and the mechanism of operation of the judge; number of bootstraps; weight and normalizer of LCI; query-fit adjustment; hysteresis and the turn estimate. The comparisons of Haya against other relevant research are performed based on searches done when this paper was being written. Future work will entail recording the parameters mentioned above, weight optimization based on data, further testing on various smartphones, different workloads and users, history re-prefill versus KV-cache migration technique [7], and having a remote model that serves as a no-swap cost variant, which is an expansion to the offline policy, as the tier policy has already been addressed by HybridInfer [22].

VII. CONCLUSION

All the mentioned four components – per candidate LCI calculation, query matching, memory admission, and amortization of the cost of model loading and history

reconstruction – are incorporated into one swap policy by the Haya approach. In contrast with Complexity-Only approach that implies 200 prompt replay, the Haya approach achieves 29.6% lower latency and 17.2% lower power, while the number of hand-offs is reduced by 70%. Moreover, the average quality in case of Haya approach is 0.746, whereas, for other approaches, it is 0.724 and 0.718 respectively. The establishment of the history tokens for 1,024 tokens constitutes 85% of the total hand-off time. It is worth mentioning that the obtained results are relevant for two different devices with replay, and thus, can only prove the effectiveness of the proposed swap-aware approach.

ACKNOWLEDGEMENT

The wording of this paper was written by the authors by hand. For grammar purposes, it has been rephrased with the help of AI.

REFERENCES

[1] W. Yin, M. Xu, Y. Li, and X. Liu, “LLM as a system service on mobile devices,” arXiv:2403.11805, 2024.
[2] J. Xiao, Q. Huang, X. Chen, and C. Tian, “Understanding large language models in your pockets: Performance study on COTS mobile devices,” arXiv:2410.03613, 2024.
[3] E. Ehsani, L. Giamattei, I. Malavolta, and R. Pietrantuono, “Sustainability is not linear: Quantifying performance, energy, and privacy trade-offs in on-device intelligence,” arXiv:2603.26603, 2026.
[4] É. Guégain and T. Coignion, “The battery price of edge AI: A study of the environmental impact of LLM inference on mobile devices,” arXiv:2609.11940, 2026.
[5] R. Bao, N. Xue, Y. Sun, and Z. Chen, “Dynamic quality-latency aware routing for LLM inference in wireless edge-device networks,” arXiv:2508.11291, 2025.
[6] Y. Zhang *et al.*, “MTRouter: Cost-aware multi-turn LLM routing with history-model joint embeddings,” in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2026, arXiv:2604.23530.
[7] T. Heo *et al.*, “Cross-model KV cache transfer in LLM families: A closed-form linear mapping for prefill reuse,” arXiv:2608.03893, 2026.
[8] O. Sanseviero and I. Ballantyne, “Introducing Gemma 3n: The developer guide,” Google Developers Blog, Jun. 26, 2025. [Online]. Available: https://developers.googleblog.com/en/introducing-gemma-3n-developer-guide/
[9] Y. Yan, J. Shen, X. Luo, and Y. Zhou, “EdgeFlow: Fast cold starts for LLMs on mobile devices,” arXiv:2604.09083, 2026.
[10] W. Yin, R. Yi, D. Xu, G. Huang, M. Xu, and X. Liu, “Elastic on-device LLM service,” in *Proc. 31st Annu. Int. Conf. Mobile Comput. Netw. (MobiCom)*, Hong Kong, China, Nov. 2025, doi: 10.1145/3680207.3765259.
[11] Z. Liu *et al.*, “MobileLLM: Optimizing sub-billion parameter language models for on-device use cases,” arXiv:2402.14905, 2024.
[12] F. Tan *et al.*, “MobileQuant: Mobile-friendly quantization for on-device language models,” arXiv:2408.13933, 2024.
[13] K. Alizadeh *et al.*, “LLM in a flash: Efficient large language model inference with limited memory,” arXiv:2312.11514, 2023.
[14] Y. Song, Z. Mi, H. Xie, and H. Chen, “PowerInfer: Fast large language model serving with a consumer-grade GPU,” arXiv:2312.12456, 2023.
[15] Z. Xue, Y. Song, Z. Mi, X. Zheng, Y. Xia, and H. Chen, “PowerInfer-2: Fast large language model inference on a smartphone,” arXiv:2406.06282, 2024.
[16] H. Yu, M. Kim, J. Park, and K. Lee, “mzCache: On-device LLM memory management under multitasking,” *Proc. ACM Int. Conf. Mobile Comput. Netw. (MobiCom)*, 2026, arXiv:2609.01338, doi: 10.1145/3795866.3844495.
[17] I. Ong *et al.*, “RouteLLM: Learning to route LLMs with preference data,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025, arXiv:2406.18665.
[18] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” *Trans. Mach. Learn. Res.*, 2024, arXiv:2305.05176.
[19] D. Ding *et al.*, “Hybrid LLM: Cost-efficient and quality-aware query routing,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024, arXiv:2404.14618.
[20] H. Jain, D. Pandey, and K. Vaidhyanathan, “CALM: A self-adaptive orchestration approach for QoS-aware routing in small language model based systems,” in *Proc. Int. Symp. Softw. Eng. Adapt. Self-Manag. Syst. (SEAMS)*, Rio de Janeiro, Brazil, Apr. 2026, arXiv:2602.03632.
[21] A. Matathammal, K. Gupta, L. Lavanya, A. V. Halgatti, P. Gupta, and K. Vaidhyanathan, “EdgeMLBalancer: A self-adaptive approach for dynamic model switching on resource-constrained edge devices,” arXiv:2502.06493, 2025.
[22] S. Koul, “HybridInfer: Thermal-aware reinforcement-learning tier routing for on-device, edge, and cloud LLM inference,” arXiv:2609.30270, 2026.
[23] A. Kabir, J. Xue, M. Zheng, and Q. Lou, “HW-Router: Hardware-aware routing for scalable multi-LLM serving,” arXiv:2608.14575, 2026.
[24] M. A. R. Siddiqui, D. Rojas, C. Yang, W. Cui, Y. Shi, and Y. Chen, “Measured joules, learned routes: Learning to route for energy-efficient LLM serving,” arXiv:2609.23085, 2026.
[25] B. Zou *et al.*, “EnerInfer: Energy-aware on-device LLM inference,” arXiv:2606.23001, 2026.
[26] E. Kim, Y. J. Jeon, and M. Han, “MemSpec: Memory-aware run-time for adaptive draft scheduling in speculative decoding on edge devices,” in *Proc. 27th ACM SIGPLAN/SIGBED Int. Conf. Lang., Compilers, Tools Embedded Syst. (LCTES)*, Jun. 2026, pp. 180–192, doi: 10.1145/3814943.3816174, arXiv:2608.10362.
[27] Google, “PowerManager: getThermalHeadroom(int forecastSeconds),” Android Developers. Accessed: Oct. 4, 2026. [Online]. Available: https://developer.android.com/reference/android/os/PowerManager
[28] realme, “realme GT 6T specifications.” Accessed: Oct. 6, 2026. [Online]. Available: https://www.realme.com/in/realme-gt-6t/specs
[29] realme, “realme 11 Pro+ specifications.” Accessed: Oct. 6, 2026. [Online]. Available: https://www.realme.com/sg/realme-11-pro-plus/specs
[30] Qwen *et al.*, “Qwen2.5 technical report,” arXiv:2412.15115, 2024.
[31] Gemma Team *et al.*, “Gemma 2: Improving open language models at a practical size,” arXiv:2408.00118, 2024.
[32] M. Abdin *et al.*, “Phi-3 technical report: A highly capable language model locally on your phone,” arXiv:2404.14219, 2024.
[33] ggml-org, “llama.cpp: LLM inference in C/C++,” GitHub repository. Accessed: Oct. 6, 2026. [Online]. Available: https://github.com/ggml-org/llama.cpp
[34] ggml-org, “GGUF file format specification,” GitHub repository. Accessed: Oct. 6, 2026. [Online]. Available: https://github.com/ggml-org/ggml/blob/master/docs/gguf.md
[35] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, arXiv:2306.05685.
[36] J. Cohen, “A coefficient of agreement for nominal scales,” *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, 1960, doi: 10.1177/001316446002000104.

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text

Match Groups

9 Not Cited or Quoted 2%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
2% Publications
1% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **9** Not Cited or Quoted 2%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
2% Publications
1% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
arxiv.org		<1%
2	Internet	
www.thinkmind.org		<1%
3	Publication	
Minsu Kim, Pinyarash Pinyoanuntapong, Bongho Kim, Walid Saad, Doru Calin. "Ed...		<1%
4	Internet	
www.generalcompute.com		<1%
5	Publication	
P. Anitha, S. Vinoth Kumar. "Enhanced Bio-Inspired Energy Efficient Security Guar...		<1%
6	Publication	
Mengmei Mei, He Huang, Ziyan Wu, Hongchuan Cheng, Haifeng Yang. "Deformati...		<1%